Agent Algorithm Flow

Table of Contents

1. Project Overview
   - 1.1 Project Origin and Nature
   - 1.2 Tech Stack
   - 1.3 Core Features Overview
2. Overall Architecture
   - 2.1 System Layered Architecture
   - 2.2 Core Data Flow
   - 2.3 Task Type System
3. Entry Point and Startup Flow
   - 3.1 CLI Argument Parsing
   - 3.2 Bootstrap Initialization Sequence
   - 3.3 Main Application Mounting
4. Agent Core Type System
   - 4.1 Agent Definition
   - 4.2 Agent Tool Call Results
   - 4.3 Task State Machine
5. Query Engine - Agent Decision Core Loop
   - 5.1 Core File Locations
   - 5.2 query() Entry Function
   - 5.3 QueryParams Parameter Structure
   - 5.4 queryLoop State Machine
   - 5.5 Compaction Boundary Cross-turn Tracking
   - 5.6 API Call Configuration Snapshot
   - 5.7 Thinking Block Rules
   - 5.8 max_output_tokens Error Recovery
   - 5.9 QueryEngine.submitMessage() Details
   - 5.10 StreamEvent Event Type Complete List
6. Tool System
   - 6.1 Tool Definition Interface
   - 6.2 Tool Registry
   - 6.3 Tool Name Constants
   - 6.4 Core Tool List
7. AgentTool - Sub-Agent Launch Core
   - 7.1 Tool Definition
   - 7.2 Agent Type Registry
   - 7.3 Built-in Agent Type Constants
   - 7.4 Validation and Error Handling
   - 7.5 Async Agent Registration Flow

- 7.6 Permission Modes

8. Built-in Agents
    - 8.1 General Purpose Agent
    - 8.2 Explore Agent (One-shot)
    - 8.3 Plan Agent (One-shot)
    - 8.4 Verification Agent (Experimental)
    - 8.5 Claude Code Guide Agent
9. runAgent - Local Agent Execution Engine
    - 9.1 Complete Function Signature
    - 9.2 System Prompt Construction
    - 9.3 enhanceSystemPromptWithEnvDetails
    - 9.4 Tool Filtering Logic
    - 9.5 Core query Call
    - 9.6 Progress Tracking and TTFT Metrics
    - 9.7 Metadata Writing
10. LocalAgentTask - Async Agent Lifecycle Management
    - 10.1 Complete State Structure
    - 10.2 Async Agent Registration
    - 10.3 AbortController Hierarchy
    - 10.4 Progress Tracker
    - 10.5 Task Notification Format
    - 10.6 Task Output to Disk
11. RemoteAgent and InProcessTeammate
    - 11.1 RemoteAgentTask
    - 11.2 InProcessTeammateTask
12. StreamingToolExecutor - Tool Parallel Orchestration Engine
    - 12.1 Core Design Principles
    - 12.2 Core Data Structures
    - 12.3 Concurrency Safety Determination
    - 12.4 Execution Queue Algorithm
    - 12.5 Single Tool Execution Flow
    - 12.6 Error Propagation and Sibling Process Abort
    - 12.7 Result Aggregation
13. Multi-Agent Coordination Mode (Coordinator)
    - 13.1 Enable Conditions
    - 13.2 Coordinator System Prompt (Complete Structure)
    - 13.3 Coordinator Tool Context
    - 13.4 Worker Tool Whitelist
    - 13.5 Internal Tools (Invisible to Workers)
14. Context Compaction System
    - 14.1 Compaction Strategy System
    - 14.2 Precise Thresholds

- 14.3 Auto Compaction Threshold Calculation
- 14.4 Compaction Execution Algorithm
- 14.5 Tombstone Mechanism After Compaction
- 14.6 Compaction Circuit Breaker
- 14.7 Compaction-triggered Transcript Flushing

15. Memory System
    - 15.1 Memory Types
    - 15.2 Memory Directory Structure
    - 15.3 Memory File Format
    - 15.4 Memory Loading Flow
    - 15.5 Relationship Between Memory and System Prompt
    - 15.6 Team Memory
16. Permissions System
    - 16.1 Permission Types
    - 16.2 Permission Levels
    - 16.3 Permission Check Flow
    - 16.4 Agent Permission Modes
    - 16.5 Permission Denial Tracking
17. State Management (AppStateStore)
    - 17.1 Complete State Structure
    - 17.2 State Update Patterns
    - 17.3 Team Context State
18. Slash Command System
    - 18.1 Command Registration
    - 18.2 Main Command List
    - 18.3 Command Execution Flow
19. API Call Layer
    - 19.1 Claude API Client
    - 19.2 Model Configuration
    - 19.3 Token Budget Management
20. Key Algorithm Flow Diagrams
    - 20.1 Complete Query Loop Flow
    - 20.2 Tool Parallel/Serial Decision Flow
    - 20.3 Multi-Agent Coordination Flow
21. Summary
    - 21.1 Agent Algorithm Core Points
    - 21.2 Key File Index
    - 21.3 Notable Engineering Designs

---

1. Project Overview

1.1 Project Origin and Nature

This project is a reverse-engineered TypeScript source code snapshot extracted from npm package source map files (`cli.js.map`), corresponding to version `@anthropic-ai/claude-code@2.1.88` . The source code was compiled by TypeScript and bundled by a bundler into a single CLI file, and the original TypeScript source code can be restored by parsing the source map, stored in the `restored-src/src/` directory (containing 1884 `.ts` / `.tsx` files).

This is not an actively developed repository, but rather a source code snapshot of a published package. However, by analyzing its structure, one can fully understand the engineering implementation of Claude Code.

1.2 Tech Stack

Layer	Technology
Runtime	Node.js / Bun
Language	TypeScript + JSX
CLI TUI	Ink (React for CLI)
AI Interface	Anthropic Claude API (version 2023-06-01)
Protocol	MCP (Model Context Protocol)
State Management	React Context + Custom Store
Experimental Features	Statsig Feature Gates (A/B Testing Framework)

1.3 Core Features Overview

- **Multi-model Support**: Sonnet / Opus / Haiku, with fallback model degradation
- **Streaming Responses**: End-to-end API streaming, reduced TTFT (Time to First Token)
- **40+ Built-in Tools**: File system, Shell, Search, MCP, Code, LSP, etc.
- **Sub-Agent System**: Supports local subprocess agents, remote agents, in-process Teammates
- **Multi-Agent Coordination**: One-to-many Worker collaboration in Coordinator mode
- **Context Compaction**: Auto Compact / Reactive Compact / History Snip multi-level compaction -> Reactive Compact is currently a closed feature
- **Memory System**: Cross-session persistent memory (four types: user / feedback / project / reference)
- **Permission Management**: Fine-grained command-level permission control
- **Structured Output**: Supports structured results via `SYNTHETIC_OUTPUT_TOOL_NAME` tool

Note

Remote agents are sub-agents running on Anthropic's cloud

- Main agent can only run locally
- Sub-agents it spawns may run locally or in the cloud

	远程	本地
关键词	长、独立、需要隔离、耗时	快、交互、需要本地状态
例子	跑测试套件、迁移脚本、批量重构	看 git 状态、改代码、发 PR

Decision conditions:

- AgentTool is the sole distribution point for local/remote
  - isolation: 'remote' → calls teleportToRemote() → task sent to CCR cloud for execution
  - Not passed / isolation: 'worktree' → runs in local REPL process

- Only two exceptions bypass AgentTool and directly call teleportToRemote(): /ultrareview and /ultraplan

---

2. Overall Architecture

2.1 System Layered Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                    CLI Entry Layer (entrypoints/)                 │
│    cli.tsx → Parse CLI args → Dispatch to different modes         │
└──────────────────────────────────────────────────────────────────┘
                                │
                                │
        ┌───────────────────────┼────────────────────────┐
        ▼                       ▼                        ▼

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│  --dump-prompts  │   │  --agent-worker= │   │  Standard        │
│    (Dump Prompts)│   │  (Agent Worker)  │   │  Interactive Mode │
└──────────────────┘   └──────────────────┘   └──────────────────┘
                                │                      │
                                │                      │
                                ▼                      ▼

                     ┌──────────────────┐   ┌──────────────────────┐
                     │ Bootstrap Layer  │   │  main.tsx (Ink/React)│
                     │ init.ts          │   │   TUI Rendering      │
                     │ setup.ts         │   └──────────┬───────────┘
                     └──────────┬───────┘              │
                                │                      ▼
              ┌─────────────────┼──────────────────────────────┐
              │                 │                              │
              ▼                 ▼                              ▼

    ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
    │ MCP Servers      │   │  AppStateStore   │   │  QueryEngine     │
    │ (Service Layer)  │   │  (Global State)  │   │  (Decision Core)  │
    └──────────────────┘   └──────────────────┘   └──────────┬───────┘
                                                             │
              ┌──────────────────────┼──────────────────────┼──────────┐
              │                      │                      │
              ▼                      ▼                      ▼

    ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
    │ StreamingTool    │   │ API Client       │   │ Compaction       │
    │ Executor         │   │ (Claude API)     │   │ Service          │
    │ (Tool Orchestration)│                  │   │                  │
    └──────────┬───────┘   └──────────────────┘   └──────────────────┘
               │
    ┌──────────┴──────────────────────────────────────────┐
    │                                                     │
    ▼                                                     ▼

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ File System Tools│   │ Shell Tools      │   │ MCP Protocol Tools│
│ Read/Write/Edit  │   │ Bash/PowerShell  │   │ MCP Tool         │
│ Glob/Grep        │   │                  │   │                  │
└──────────────────┘   └──────────────────┘   └──────────────────┘
```

2.2 Core Data Flow

```
User Input
    │
    ▼
submitMessage() → query() [AsyncGenerator]
    │
    ├── yield StreamEvent / Message  (Streaming Events)
    │       ├── text delta          (Text Fragment)
    │       ├── tool_use block      (Tool Call Request)
    │       ├── tool_result         (Tool Execution Result)
    │       └── message_update      (Status Update)
    │
    └── Return Terminal (End State)
```

2.3 Task Type System

Files: `types/Task.ts` and various Task implementation files

```
// Complete enumeration of Task types
type TaskType =
  | 'local_agent'        // Local sub-agent (started via AgentTool)
  | 'local_bash'         // Local Shell command
  | 'remote_agent'       // Remote agent in CCR environment
  | 'in_process_teammate' // In-process Teammate (memory-level, no subprocess)
  | 'local_workflow'     // Local workflow script
  | 'monitor_mcp'        // MCP monitoring task
  | 'dream'              // Dream / autonomous exploration mode
```

Dream / Auto Dream:

- Dream is Claude Code's automatic memory consolidation mechanism — it runs a sub-agent silently in the background to periodically organize and update memory files
- Workflow:
  - Orient — View memory directory structure, read entry index, understand existing memories
  - Gather — Collect new signals from daily logs, conflicting facts, conversation records
  - Consolidate — Write/update memory files, merge duplicates
  - Prune — Trim the entry index (memdir.md), delete outdated memories, fix conflicts
- Trigger conditions:

门控	条件	说明
特性开关	GrowthBook tengu_onyx_plover 或用户设置 autoDreamEnabled	总开关
时间门	距上次整合 ≥ minHours（默认 24h）	避免过于频繁
扫描节流	上次扫描 ≥ 10 分钟	防止时间门通过但会话门未通过时重复触发
会话门	新增会话数 ≥ minSessions（默认 5 个）	确保有足够素材可整合
锁文件	无其他进程正在整合（PID 文件 + 过期 1h 自动回收）	防并发

Each Task type corresponds to an independent Task component file, uniformly managed through the `AppStateStore.tasks` dictionary for lifecycle management.

3. Entry Point and Startup Flow

3.1 CLI Argument Parsing

File: `entrypoints/cli.tsx`

```
// Entry dispatch logic
async function main() {
  const parsed = parseArgs(process.argv);

  // Fast path: version number
  if (parsed.version || parsed.v) return printVersion();

  // Debug/dump path: print rendered system prompts
  if (parsed.dumpPrompts) {
    return createDumpPromptsFetch(parsed.dumpPrompts)();
  }

  // Agent Worker mode: entry point for sub-agents
  // Startup args: --agent-worker=<taskId> --task-script=<path>
  if (parsed.agentWorker) {
    return runAgentWorker({
      taskId: parsed.agentWorker,
      taskScript: parsed.taskScript,
    });
  }

  // Daemon worker mode
  if (parsed.daemonWorker) {
    return daemonWorkerLoop();
  }

  // MCP server mode
  if (parsed.mcpServerMode) {
    return runMCPServer(parsed);
  }

  // Standard interactive mode
  return runClaudeCodeCLI(parsed);
}
```

3.2 Bootstrap Initialization Sequence

File: `bootstrap/init.ts`

```
init()
  │
  ├── Detect CLI entry point (sdk-ts / sdk-py / sdk-cli / null)
  │     └─ Affects built-in agent loading strategy (SDK mode can disable built-in agents)
  │
  ├── Load Claude Code data directory ~/.claude/
  │     ├── settings.json
  │     ├── memory/  (Memory files)
```

```
│      ├── commands/ (Custom commands)
│      └── skills/   (Skill definitions)
│
├── Detect runtime mode
│      ├── CLAUDE_CODE_REMOTE=1 → RemoteAgentTask starts
│      ├── CLAUDE_CODE_COORDINATOR_MODE=1 → Coordinator starts
│      └── Default → Main interactive session
│
├── Initialize MCP servers
│      ├── Check MCP_SERVERS environment variable
│      ├── Check ~/.claude/mcp_servers.json
│      └── Start MCP connections (wait up to 30s)
│
└── Initialize Statsig GrowthBook (A/B testing)
       └ Fetch experimental feature flags: COORDINATOR_MODE / REACTIVE_COMPACT, etc.
```

> **Note**
>
> The Bootstrap layer is the mechanism by which Claude Code pulls **startup configuration** from Anthropic's servers at launch and persists it locally, ensuring the application can immediately get the latest server-side configuration after startup

3.3 Main Application Mounting

File: `main.tsx` (Compiled large file, ~803KB)

Uses **Ink** (React for CLI) to render TUI:

```
<AppStateStore.Provider initialState={appState}>
  <ThemeProvider>
    <ScreenRouter>
      {/* Main interactive screen */}
      <MainScreen />
      {/* Config editing screen */}
      <ConfigScreen />
      {/* Help screen */}
      <HelpScreen />
      {/* Agent Team view */}
      <TeamScreen />
    </ScreenRouter>
  </ThemeProvider>
</AppStateStore.Provider>
```

---

4. Agent Core Type System

4.1 Agent Definition

File: `tools/AgentTool/loadAgentsDir.ts`

```
// Core definition of an Agent
interface AgentDefinition {
  agentType: string;           // Unique identifier, e.g., "general-purpose"
```

```
  whenToUse?: string;           // Agent selection hint (for LLM to understand when to use)
  tools: string[] | ['*'];      // Available tool list, '*' means all
  source: 'built-in' | string;  // Source: built-in / user-defined directory
  baseDir: string;              // Directory where the definition file is located
  model?: string;               // Optional: specified model
  maxTurns?: number;            // Maximum conversation turns
  permissionMode?: PermissionMode; // Permission mode

  // Factory function: dynamically construct system prompt
  getSystemPrompt(
    context: { toolUseContext: Pick<ToolUseContext, 'options'> }
  ): string;
}
```

Context that needs dynamic loading:

动态加载的 12 类上下文

#	上下文类型	来源	缓存策略
1	**技能/工具使用策略**	skillToolCommands + enabledTools	缓存
2	**记忆内容**	.claude/memdir/ 目录	缓存
3	**环境信息**	实时读取：cwd、git状态、平台、shell、OS、模型ID	缓存
4	**语言偏好**	settings.json → language	缓存
5	**输出风格配置**	output style 插件	缓存
6	**MCP 服务器指令**	已连接 MCP server 的 instructions	**每轮重算**（MCP可动态连接/断开）
7	**Scratchpad 目录**	settings.json → permissions.scratchpad	缓存
8	**模型知识截止日期**	根据模型 ID 映射	缓存
9	**Token 预算指令**	GrowthBook feature flag	缓存
10	**Proactive/Brief 指南**	KAIROS 特性开关	缓存
11	**会话钩子说明**	用户配置的 hooks	缓存
12	**Agent 特定 MCP servers**	frontmatter 中定义的 MCP 配置	运行时一次性初始化

Key design notes:

- Static/dynamic separation: The static part before `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` can be used for cross-organization prompt caching (saving ~90% token cost)
- MCP special handling: MCP instructions are recalculated every turn, because MCP servers may connect/disconnect mid-conversation
- toolUseContext's role: Passes options (including mainLoopModel, mcpClients, agentDefinitions, etc.), enabling getSystemPrompt to make decisions based on runtime state

4.2 Agent Tool Call Results

File: `tools/AgentTool/AgentTool.tsx`

```
// Return result from AgentTool synchronous call
type AgentToolResult =
  | { status: 'success'; result: string }
  | { status: 'async_launched'; agentId: string; outputFile: string }
  | { status: 'error'; error: string };

// Final output of async agent
interface AsyncAgentResult {
  status: 'completed' | 'failed' | 'killed';
```

```
  summary: string;      // Task summary (used to notify parent agent)
  result?: string;       // Complete result
  usage?: {
    total_tokens: number;
    tool_uses: number;
    duration_ms: number;
  };
}
```

4.3 Task State Machine

```
// Lifecycle states of a Task
type TaskStatus =
  | 'pending'    // Registered, waiting to start
  | 'running'    // Executing
  | 'completed'  // Finished normally
  | 'failed'     // Execution failed
  | 'killed'     // Manually stopped
  | 'paused';    // Paused (waiting for user confirmation)

// LocalAgentTaskState complete structure
interface LocalAgentTaskState extends TaskStateBase {
  type: 'local_agent';
  agentId: string;                    // Agent ID
  agentType: string;                   // Agent type
  prompt: string;                      // Initial prompt
  selectedAgent?: AgentDefinition;    // Agent definition object
  model?: string;                      // Specified model
  abortController?: AbortController;   // Abort controller
  error?: string;                     // Error message
  result?: AgentToolResult;           // Final result
  progress?: AgentProgress;            // Real-time progress
  retrieved: boolean;                  // Whether output has been read
  messages?: Message[];                // Agent message history
  isBackgrounded: boolean;            // Whether running in background
  pendingMessages: string[];           // Pending message queue
  retain: boolean;                     // Whether to retain (not affected by evict)
  diskLoaded: boolean;                 // Whether loaded from disk
  evictAfter?: number;                 // Evict after N accesses
  lastReportedToolCount: number;       // Last reported tool call count
  lastReportedTokenCount: number;      // Last reported token count
}
```

---

5. Query Engine - Agent Decision Core Loop

5.1 Core File Locations

File	Lines	Role
`query.ts`	~800+	`query()` function definition + `queryLoop()` core loop
`QueryEngine.ts`	~1300+	Complete implementation of `QueryEngine` class
`query/config.ts`	-	`buildQueryConfig()` query configuration building

File	Lines	Role
`query/deps.ts`	-	`productionDeps` dependency injection
`query/stopHooks.ts`	-	Stop hook handling
`query/tokenBudget.ts`	-	Token budget management

5.2 query() Entry Function

File: `query.ts` (lines 219-239)

```
export async function* query(
  params: QueryParams,
): AsyncGenerator<
  | StreamEvent        // Streaming events (text / tool_use / tool_result, etc.)
  | RequestStartEvent   // Request start event
  | Message             // Message object
  | TombstoneMessage    // Tombstone message (placeholder for compacted history)
  | ToolUseSummaryMessage, // Tool call summary
  Terminal              // Return type
> {
  const consumedCommandUuids: string[] = [];

  // Call queryLoop, execute the actual streaming loop
  const terminal = yield* queryLoop(params, consumedCommandUuids);

  // Notify all consumed commands of completion
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed');
  }

  return terminal;
}
```

Key design: Uses TypeScript Generator + `yield*` (delegating generator) for async streaming processing. `yield*` passes values from the sub-generator directly upward until the sub-generator returns a `Terminal` object.

5.3 QueryParams Parameter Structure

```
export type QueryParams = {
  messages: Message[];              // Conversation history messages
  systemPrompt: SystemPrompt;       // System prompt (rendered)
  userContext: { [k: string]: string };    // User context (session-related)
  systemContext: { [k: string]: string };  // System context (environment-related)
  canUseTool: CanUseToolFn;          // Permission check function
  toolUseContext: ToolUseContext;    // Tool execution context
  fallbackModel?: string;           // Fallback model when API call fails
  querySource: QuerySource;           // Query source (main / agent / sdk)
  maxOutputTokensOverride?: number;  // Max output token count override
  maxTurns?: number;                // Maximum conversation turns
  skipCacheWrite?: boolean;          // Skip cache write
  taskBudget?: { total: number };    // API task_budget (beta)
```

```
  deps?: QueryDeps;                  // Dependency injection (for testability)
};
```

5.4 queryLoop State Machine

File: `query.ts` (starting at line 241)

```
type State = {
  messages: Message[];                          // Conversation history (mutable)
  toolUseContext: ToolUseContext;                 // Tool execution context
  autoCompactTracking: AutoCompactTrackingState | undefined;
  maxOutputTokensRecoveryCount: number;          // max_output_tokens recovery count
  hasAttemptedReactiveCompact: boolean;          // Whether reactive compaction has been
attempted
  maxOutputTokensOverride: number | undefined;  // Max output override
  pendingToolUseSummary: Promise<ToolUseSummaryMessage | null> | undefined;
  stopHookActive: boolean | undefined;          // Whether stop hook is active
  turnCount: number;                            // Current turn (starting from 1)
  transition: Continue | undefined;             // Why the last iteration continued
};
```

Initial state setup (lines 268-279):

```
let state: State = {
  messages: params.messages,
  toolUseContext: params.toolUseContext,
  maxOutputTokensOverride: params.maxOutputTokensOverride,
  autoCompactTracking: undefined,
  stopHookActive: undefined,
  maxOutputTokensRecoveryCount: 0,
  hasAttemptedReactiveCompact: false,
  turnCount: 1,
  pendingToolUseSummary: undefined,
  transition: undefined,
};
```

5.5 Compaction Boundary Cross-turn Tracking

```
// task_budget.remaining: Track remaining budget across compaction boundaries
// Before compaction: API server maintains its own countdown from {total}
// After compaction: server only sees summary, remaining informs it of actual consumption
before compaction
let taskBudgetRemaining: number | undefined = undefined;
```

5.6 API Call Configuration Snapshot

```
// Snapshot all immutable configurations at query entry
// Avoid re-reading environment variables on each iteration
const config = buildQueryConfig();

// buildQueryConfig snapshot contents:
// - Model configuration (main loop model / fallback model)
```

```
// - Context window size
// - API limit configuration
// - Tool list
// Note: feature() feature flags are intentionally NOT in the snapshot (read in real-time)
```

5.7 Thinking Block Rules

File: `query.ts` (lines 151-163)

```
/**
 * Rules for thinking blocks ("wizard rules"):
 * 1. Messages containing thinking / redacted_thinking blocks must have max_thinking_length
> 0
 * 2. Thinking blocks cannot be the last block in a message block
 * 3. Thinking blocks must be preserved throughout the entire assistant trace
 *    (single turn, or the turn contains tool_use + its tool_result + subsequent assistant
message)
 */
```

5.8 max_output_tokens Error Recovery

```
const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3;

// Detection: Is this a max_output_tokens error (needs delayed reporting)?
function isWithheldMaxOutputTokens(
  msg: Message | StreamEvent | undefined,
): msg is AssistantMessage {
  return msg?.type === 'assistant' && msg.apiError === 'max_output_tokens';
}

// Recovery logic:
// If max_output_tokens error received → attempt recovery → retry up to 3 times
// Retry strategy: Increase maxOutputTokensOverride (using ESCALATED_MAX_TOKENS)
```

5.9 QueryEngine.submitMessage() Details

File: `QueryEngine.ts`

```
async *submitMessage(
  prompt: string | ContentBlockParam[],
  options?: { uuid?: string; isMeta?: boolean }
): AsyncGenerator<SDKMessage, void, unknown>
```

Flow:

```
1. Append user message to mutableMessages[]
   └─ If prompt is string → wrap as text ContentBlock (structured parsing of raw content)
   └─ If prompt is ContentBlockParam[] → append directly

2. Build system prompt
   └─ fetchSystemPromptParts() gets three parts:
       - defaultSystemPrompt (static part)
```

```
        - userContext (user-related)
        - systemContext (system-related)
    └─ Merge coordinatorUserContext (if coordinator mode enabled)
    └─ Inject loadMemoryPrompt() (if custom MEM.AUTO_MEM_PATH)

3. Wrap canUseTool (permission check wrapper)
    └─ Track permissionDenials[] (SDK returns to SDK caller)

4. Call query()
    └─ Pass all constructed parameters

5. Process StreamEvent / Message
    └─ Filter API error retry events (SDKStatus)
    └─ Append TTFT metrics to toolUseContext
    └─ Update messageIndexMap
    └─ Track token usage to totalUsage

6. Process completion/error results
    └─ CompactBoundary → flush to memory transcript
    └─ error_max_turns → max turns reached
    └─ error_max_budget_usd → USD budget exceeded
    └─ error_during_execution → execution exception
```

5.10 StreamEvent Event Type Complete List

```typescript
// Text fragment
type TextEvent = { type: 'text'; content: string; isOpen: boolean };

// Tool call
type ToolUseEvent = { type: 'tool_use'; tool: ToolUseBlock };

// Tool result (for synchronous tools)
type ToolResultEvent = { type: 'tool_result'; result: ToolResultBlockParam };

// Error event
type ErrorEvent = { type: 'error'; error: string; code?: string };

// Done event
type DoneEvent = { type: 'done' };

// Hover preview (preview before tool call)
type PendingToolUseEvent = { type: 'pending_tool_use'; tool: ToolUseBlock };

// Progress update
type ProgressEvent = { type: 'progress'; progress: number };

// Speculation update (prefill / real-time completion)
type SpeculationEvent = { type: 'speculation_update'; spec: Speculation };

// API metrics
type ApiMetricsEvent = {
  type: 'message_start';
  message: AssistantMessage;
```

```
  ttftMs: number;  // Time to First Token
};
```

---

6. Tool System

> **Note**
>
> See the "Tool Analysis" document for details

6.1 Tool Definition Interface

File: `Tool.ts`

```
interface Tool {
  name: string;                        // Unique name
  description: string;                 // Description (for LLM to understand purpose)
  matchesCapability?: Capability;       // Matching capability marker
  inputSchema: ZodSchema;             // Input parameter Zod Schema

  // Permission related
  requiresPermissions?: Permission[];
  allowedWhenPaused?: boolean;         // Whether executable when paused

  // Execution entry
  invoke(input: unknown, context: ToolUseContext): Promise<ToolResult>;
  stream?(
    input: unknown,
    context: ToolUseContext
  ): AsyncGenerator<Partial<ToolResult>>; // Streaming execution

  // Optional: concurrency safety determination
  isConcurrencySafe?(
    input: unknown
  ): boolean;
}

interface ToolUseContext {
  abortController: AbortController;
  toolUseId: string;
  options: {
    cwd: string;
    env: Record<string, string>;
    sessionId: string;
    // ... more context
  };
  pushApiMetricsEntry?: (ttftMs: number) => void;
}
```

6.2 Tool Registry

```typescript
class ToolRegistry {
  // Core data structure
  private tools: Map<string, Tool> = new Map();

  // Register
  register(tool: Tool): void;

  // Lookup by name
  get(name: string): Tool | undefined;

  // Get all
  getAll(): Tool[];

  // Filter by agent (permission)
  getToolsForAgent(agentId: string, permissions: Permission[]): Tool[];

  // Filter by capability
  getToolsByCapability(capability: Capability): Tool[];
}

// Find tool (with auxiliary info)
function findToolByName(tools: Tools, name: string): Tool | undefined;
```

6.3 Tool Name Constants

File: `constants/tools.ts`

```typescript
export const FILE_READ_TOOL_NAME = 'Read';
export const FILE_WRITE_TOOL_NAME = 'Write';
export const FILE_EDIT_TOOL_NAME = 'Edit';
export const BASH_TOOL_NAME = 'Bash';
export const GREP_TOOL_NAME = 'Grep';
export const GLOB_TOOL_NAME = 'Glob';
export const TASK_CREATE_TOOL_NAME = 'TaskCreate';
export const TODO_WRITE_TOOL_NAME = 'TodoWrite';
export const AGENT_TOOL_NAME = 'Agent';
export const LEGACY_AGENT_TOOL_NAME = 'Task';
export const SKILL_TOOL_NAME = 'Skill';
export const SEND_MESSAGE_TOOL_NAME = 'SendMessage';
export const TASK_STOP_TOOL_NAME = 'TaskStop';
export const TEAM_CREATE_TOOL_NAME = 'TeamCreate';
export const TEAM_DELETE_TOOL_NAME = 'TeamDelete';
export const SYNTHETIC_OUTPUT_TOOL_NAME = 'SyntheticOutput';
```

> **Note**
>
> See the "Tool Analysis" document for details

6.4 Core Tool List

Tool	File Location	Description
`Read`	`tools/FileReadTool/`	Read files (supports line ranges, images)
`Write`	`tools/FileWriteTool/`	Write/overwrite files
`Edit`	`tools/FileEditTool/`	Precise string replacement editing
`Glob`	`tools/GlobTool/`	Match file paths by pattern
`Grep`	`tools/GrepTool/`	Content search (supports regex)
`Bash`	`tools/BashTool/`	Execute shell commands
`Agent`	`tools/AgentTool/`	Launch sub-agent (core)
`SendMessage`	`tools/SendMessageTool/`	Send messages to running agents
`TaskStop`	`tools/TaskStopTool/`	Stop agent tasks
`Skill`	`tools/SkillTool/`	Invoke Skill
`MCP`	`tools/McpTool/`	Invoke MCP tools
`WebSearch`	`tools/WebSearchTool/`	Internet search
`WebFetch`	`tools/WebFetchTool/`	Fetch web pages
`TodoWrite`	`tools/TodoWriteTool/`	Task list management
`AskUserQuestion`	`tools/AskUserQuestionTool/`	Ask user questions
`TaskCreate`	`tools/TaskCreateTool/`	Create Task
`TaskOutput`	`tools/TaskOutputTool/`	Read agent output
`TeamCreate`	`tools/TeamCreateTool/`	Create Agent Team
`Compact`	`commands/compact/`	Manually trigger compaction
`Config`	`tools/ConfigTool/`	Configuration management
`LSP`	`tools/LSPTool/`	Language Server Protocol
`NotebookEdit`	`tools/NotebookEditTool/`	Jupyter editing
`Sleep`	`tools/SleepTool/`	Delayed execution

---

7. AgentTool - Sub-Agent Launch Core

> **Note**
>
> See the sub-agent documentation for details

7.1 Tool Definition

File: `tools/AgentTool/AgentTool.tsx`

AgentTool is the entry point tool for the entire sub-agent system, defined as follows:

```
// Input Schema (Zod)
const baseInputSchema = z.object({
  description: z.string().describe('Brief description (3-5 words)'),
  prompt: z.string().describe('Agent task description'),
  subagent_type: z.string().optional(),     // Agent type (default: general-purpose)
  model: z.enum(['sonnet', 'opus', 'haiku']).optional(),  // Specified model
  run_in_background: z.boolean().optional() // Run in background
});
```

```
// Complete Schema (additional fields for Team mode)
const fullInputSchema = baseInputSchema.merge(z.object({
  name: z.string().optional(),              // Agent name (for SendMessage routing)
  team_name: z.string().optional(),        // Team name
  mode: permissionModeSchema().optional(),  // Permission mode
  isolation: z.enum(['worktree', 'remote']).optional(), // Isolation mode
  cwd: z.string().optional()               // Working directory
}));
```

7.2 Agent Type Registry

File: `tools/AgentTool/builtInAgents.ts`

```
export function getBuiltInAgents(): AgentDefinition[] {
  // SDK mode can disable all built-in agents
  if (isNonInteractiveSession &&
      CLAUDE_AGENT_SDK_DISABLE_BUILTIN_AGENTS) {
    return [];
  }

  // Coordinator mode: return coordinator agent
  if (feature('COORDINATOR_MODE') &&
      process.env.CLAUDE_CODE_COORDINATOR_MODE) {
    return getCoordinatorAgents();
  }

  const agents = [
    GENERAL_PURPOSE_AGENT,
    STATUSLINE_SETUP_AGENT,
  ];

  // Explore / Plan Agent (A/B test controlled)
  if (areExplorePlanAgentsEnabled()) {
    agents.push(EXPLORE_AGENT, PLAN_AGENT);
  }

  // Claude Code Guide (only loaded for non-SDK entry points)
  if (isNonSdkEntrypoint) {
    agents.push(CLAUDE_CODE_GUIDE_AGENT);
  }

  // Verification Agent (GrowthBook experiment)
  if (feature('VERIFICATION_AGENT') &&
      getFeatureValue('tengu_hive_evidence', false)) {
    agents.push(VERIFICATION_AGENT);
  }

  return agents;
}
```

7.3 Built-in Agent Type Constants

File: `tools/AgentTool/constants.ts`

```
// Agent tool name
export const AGENT_TOOL_NAME = 'Agent';
export const LEGACY_AGENT_TOOL_NAME = 'Task';  // Backward compatibility

// One-shot agents (parent agent will not continue interacting)
// Saves tokens: ~135 chars × 34M runs/week
export const ONE_SHOT_BUILTIN_AGENT_TYPES: ReadonlySet<string> = new Set([
  'Explore',   // One-shot exploration
  'Plan',      // One-shot planning
]);

// Verification Agent
export const VERIFICATION_AGENT_TYPE = 'verification';
```

7.4 Validation and Error Handling

File: `AgentTool.tsx` (lines 262-409)

```
// Error 1: Agent Teams not enabled
if (team_name && !isAgentSwarmsEnabled()) {
  throw new Error('Agent Teams is not yet available on your plan.');
}

// Error 2: Teammate cannot spawn other Teammates
if (isTeammate() && teamName && name) {
  throw new Error('Teammates cannot spawn other teammates...');
}

// Error 3: In-process Teammate cannot spawn background agents
if (isInProcessTeammate() && teamName && run_in_background === true) {
  throw new Error('In-process teammates cannot spawn background agents...');
}

// Error 4: Agent type not found
throw new Error(`Agent type '${effectiveType}' not found. ` +
  `Available agents: ${agents.map(a => a.agentType).join(', ')}`);

// Error 5: Agent denied by permission rule
throw new Error(`Agent type '${effectiveType}' has been denied by permission rule...`);

// MCP server dependency check (lines 369-409)
// Wait up to 30 seconds for MCP servers to be ready (POLL_INTERVAL_MS = 500ms)
if (!hasRequiredMcpServers(selectedAgent, serversWithTools)) {
  throw new Error(`Agent '${selectedAgent.agentType}' requires MCP servers...`);
}
```

7.5 Async Agent Registration Flow

File: `AgentTool.tsx` (lines 686-764)

```
// shouldRunAsync decision
// Rules: run_in_background=true or Team mode → async
```

```
if (shouldRunAsync) {
  // 1. Register async agent, get taskId
  const agentBackgroundTask = registerAsyncAgent({
    agentId: asyncAgentId,
    description,
    prompt,
    selectedAgent,
    setAppState: rootSetAppState,
    toolUseId: toolUseContext.toolUseId,
  });

  // 2. Register name routing (for SendMessage)
  if (name) {
    rootSetAppState(prev => {
      const next = new Map(prev.agentNameRegistry);
      next.set(name, asAgentId(asyncAgentId));
      return { ...prev, agentNameRegistry: next };
    });
  }

  // 3. Return async launch result
  return {
    data: {
      status: 'async_launched',
      agentId: agentBackgroundTask.agentId,
      description,
      prompt,
      outputFile: getTaskOutputPath(agentBackgroundTask.agentId),
      canReadOutputFile,
    }
  };
} else {
  // Sync mode: run agent directly in current query
  return { data: { status: 'sync', result: await runAgentSync(...) } };
}
```

7.6 Permission Modes

```
type PermissionMode =
  | 'acceptEdits'    // Accept edits (default)
  | 'restricted'    // Restricted mode
  | 'browseOnly'     // Browse only
  | 'modes.loco';   // Custom mode

// Auto-authorization strategy for permission modes
// - async Agent with prompts allowed: canShowPermissionPrompts = true
// - bubble mode: canShowPermissionPrompts = false
// - sync Agent: canShowPermissionPrompts = true (default)
```

---

8. Built-in Agents

> **Note**
>
> - The AgentTool chapter focuses on how SubAgents are spawned
> - The Built-in Agent chapter focuses on how these built-in SubAgents are specifically configured

8.1 General Purpose Agent

File: `tools/AgentTool/built-in/generalPurposeAgent.ts`

```
const SHARED_PREFIX = `You are an agent for Claude Code,
Anthropic's official CLI for Claude. Given the user's message,
you should use the tools available to complete the task.
Complete the task fully—don't gold-plate, but don't leave it half-done.`;

const SHARED_GUIDELINES = `Your strengths:
- Searching for code, configurations, and patterns across large codebases
- Analyzing multiple files to understand system architecture
- Investigating complex questions that require exploring many files
- Performing multi-step research tasks

Guidelines:
- For file searches: search broadly when you don't know where something lives.
- For analysis: Start broad and narrow down.
- Be thorough: Check multiple locations, consider different naming conventions.
- NEVER create files unless absolutely necessary. Prefer editing existing files.
- NEVER proactively create documentation files (*.md) unless explicitly requested.`;

export const GENERAL_PURPOSE_AGENT: BuiltInAgentDefinition = {
  agentType: 'general-purpose',
  whenToUse:
    'General-purpose agent for researching complex questions, ' +
    'searching for code, and executing multi-step tasks. When you are ' +
    'searching for a keyword or file and are not confident that you will ' +
    'find the right match in the first few tries use this agent.',
  tools: ['*'],       // All tools
  source: 'built-in',
  baseDir: 'built-in',
  model: undefined,   // Use getDefaultSubagentModel() default value
  getSystemPrompt: getGeneralPurposeSystemPrompt,
};
```

8.2 Explore Agent (One-shot)

```
export const EXPLORE_AGENT = {
  agentType: 'Explore',
  whenToUse: 'Use when searching for a keyword or file...',
  tools: ['Glob', 'Grep', 'Read', 'Bash'],
  source: 'built-in',
  getSystemPrompt: () => `You are a codebase exploration agent...`,
};
```

```
// Feature: ONE_SHOT_BUILTIN_AGENT_TYPES member, parent agent will not continue interaction
(i.e., "one-shot")
```

8.3 Plan Agent (One-shot)

```
export const PLAN_AGENT = {
  agentType: 'Plan',
  // Used for planning mode
  tools: ['*'],
  source: 'built-in',
  getSystemPrompt: () => `You are a planning agent...`,
};
// Feature: ONE_SHOT_BUILTIN_AGENT_TYPES member
```

8.4 Verification Agent (Experimental)

```
export const VERIFICATION_AGENT = {
  agentType: 'verification',
  tools: ['Bash', 'Read', 'Grep'],
  source: 'built-in',
  getSystemPrompt: () => `You are a code verification agent...`,
};
// Feature: Controlled by GrowthBook experiment 'tengu_hive_evidence'
```

System prompt in main agent for invoking this verification agent (not yet enabled):

```
  getFeatureValue_CACHED_MAY_BE_STALE('tengu_hive_evidence', false)
    ? `### **The Contract**

    When **non-trivial implementation** happens on your turn, independent adversarial
verification **must** happen before you report completion — regardless of who did the
implementing (you directly, a fork you spawned, or a subagent).

    **You are the one reporting to the user; you own the gate.**

    - **Non-trivial means:** 3+ file edits, backend/API changes, or infrastructure changes.

    - **Action:** Spawn the `${AGENT_TOOL_NAME}` tool with
`subagent_type="${VERIFICATION_AGENT_TYPE}"`.

    - **Requirements:** * Your own checks, caveats, and a fork's self-checks do **NOT**
substitute.

        - Only the verifier assigns a verdict; you cannot self-assign `PARTIAL`.

        - Pass the original user request, all files changed (by anyone), the approach, and
the plan file path if applicable.

        - Flag concerns if you have them, but do **NOT** share test results or claim things
work.
```

```
    **Handling Results:**

    - **On FAIL:** Fix, resume the verifier with its findings plus your fix, and repeat
until `PASS`.

    - **On PASS:** Spot-check it — re-run 2-3 commands from its report. Confirm every `PASS`
has a Command run block with output that matches your re-run. If any `PASS` lacks a command
block or diverges, resume the verifier with the specifics.

    - **On PARTIAL:** (From the verifier) Report what passed and what could not be
verified.`

    : null,
```

8.5 Claude Code Guide Agent

```
export const CLAUDE_CODE_GUIDE_AGENT = {
  agentType: 'claude-code-guide',
  tools: ['WebSearch', 'WebFetch'],
  source: 'built-in',
  getSystemPrompt: () => `You are a guide for Claude Code users...`,
};
// Feature: Only loaded for non-SDK entry points (isNonSdkEntrypoint check)
```

---

9. runAgent - Local Agent Execution Engine

9.1 Complete Function Signature

File: `tools/AgentTool/runAgent.ts` (lines 248-329)

```
export async function* runAgent({
  agentDefinition,          // Agent definition
  promptMessages,           // Initial messages (optional)
  toolUseContext,            // Tool execution context
  canUseTool,               // Permission check function
  isAsync,                  // Whether async mode
  canShowPermissionPrompts, // Whether to show permission dialogs
  forkContextMessages,       // Fork context messages
  querySource,              // Query source
  override,                 // Override parameters
  model,                    // Specified model
  maxTurns,                 // Maximum turns
  preserveToolUseResults,   // Preserve tool results
  availableTools,           // Available tool list
  allowedTools,             // Allowed tool list
  onCacheSafeParams,        // Cache-safe params callback
  contentReplacementState,  // Content replacement state
  useExactTools,            // Use exact tool list
  worktreePath,             // Git worktree path
  description,              // Task description
  transcriptSubdir,         // Transcript subdirectory
```

```
  onQueryProgress,          // Progress callback
}): AsyncGenerator<Message, void>
```

9.2 System Prompt Construction

File: `runAgent.ts` (lines 906-932)

```
async function getAgentSystemPrompt(
  agentDefinition: AgentDefinition,
  toolUseContext: Pick<ToolUseContext, 'options'>,
  resolvedAgentModel: string,
  additionalWorkingDirectories: string[],
  resolvedTools: readonly Tool[],
): Promise<string[]> {
  const enabledToolNames = new Set(resolvedTools.map(t => t.name));

  try {
    // Prefer the agent's own getSystemPrompt
    const agentPrompt = agentDefinition.getSystemPrompt({ toolUseContext });
    const prompts = [agentPrompt];

    return await enhanceSystemPromptWithEnvDetails(
      prompts,
      resolvedAgentModel,
      additionalWorkingDirectories,
      enabledToolNames,
    );
  } catch (_error) {
    // Fallback: use default agent prompt
    return enhanceSystemPromptWithEnvDetails(
      [DEFAULT_AGENT_PROMPT],
      resolvedAgentModel,
      additionalWorkingDirectories,
      enabledToolNames,
    );
  }
}
```

9.3 enhanceSystemPromptWithEnvDetails

```
async function enhanceSystemPromptWithEnvDetails(
  existingSystemPrompt: string[],
  model: string,
  additionalWorkingDirectories?: string[],
  enabledToolNames?: Set<string>,
): Promise<string[]> {
  const prompts = [...existingSystemPrompt];

  // 1. Add working directory comments (no emoji, use absolute paths)
  if (additionalWorkingDirectories?.length) {
    prompts.push(
      `Additional working directories: ${additionalWorkingDirectories.join(', ')}`
    );
```

```
  }

  // 2. Add DiscoverSkills guidance (if enabled)
  if (isDiscoverSkillsEnabled()) {
    prompts.push(getDiscoverSkillsSystemPrompt());
  }

  // 3. Add environment info
  prompts.push(await computeEnvInfo());

  return prompts;
}
```

9.4 Tool Filtering Logic

```
// Lines 500-502
const resolvedTools = useExactTools
  ? availableTools  // Use passed exact tool list
  : resolveAgentTools(
      agentDefinition,
      availableTools,
      isAsync,
    ).resolvedTools;  // Resolve tool list according to agent definition

// resolveAgentTools implementation:
// 1. If agentDefinition.tools === ['*'] → return all availableTools
// 2. Otherwise → filter to tools in agentDefinition.tools
// 3. If isAsync → additionally apply ASYNC_AGENT_ALLOWED_TOOLS restrictions
```

9.5 Core query Call

```
// Lines 748-756
for await (const message of query({
  messages: initialMessages,
  systemPrompt: agentSystemPrompt,
  userContext: resolvedUserContext,
  systemContext: resolvedSystemContext,
  canUseTool,
  toolUseContext: agentToolUseContext,
  querySource,
  maxTurns: maxTurns ?? agentDefinition.maxTurns,
})) {
  // Process each message event
}
```

9.6 Progress Tracking and TTFT Metrics

```
// Lines 758-768
onQueryProgress?.();  // Notify parent of progress update

// Forward TTFT (Time to First Token) metrics to parent agent
if (message.type === 'stream_event' &&
```

```
    message.event.type === 'message_start' &&
    message.ttftMs != null) {
  toolUseContext.pushApiMetricsEntry?.(message.ttftMs);
  continue;
}
```

9.7 Metadata Writing

```
// Lines 735-742
// Before query loop: write agent metadata to disk

void recordSidechainTranscript(initialMessages, agentId).catch(...);

void writeAgentMetadata(agentId, {
  agentType: agentDefinition.agentType,
  ...(worktreePath && { worktreePath }),
  ...(description && { description }),
}).catch(...);

// Track last recorded message UUID (for transcript append)
let lastRecordedUuid: UUID | null =
  initialMessages.at(-1)?.uuid ?? null;
```

---

10. LocalAgentTask - Async Agent Lifecycle Management

10.1 Complete State Structure

File: `tasks/LocalAgentTask/LocalAgentTask.tsx` (lines 116-148)

```
export type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent';
  agentId: string;
  prompt: string;
  selectedAgent?: AgentDefinition;
  agentType: string;
  model?: string;
  abortController?: AbortController;
  unregisterCleanup?: () => void;
  error?: string;
  result?: AgentToolResult;
  progress?: AgentProgress;
  retrieved: boolean;
  messages?: Message[];
  lastReportedToolCount: number;
  lastReportedTokenCount: number;
  isBackgrounded: boolean;
  pendingMessages: string[];
  retain: boolean;
  diskLoaded: boolean;
  evictAfter?: number;
};
```

10.2 Async Agent Registration

File: `tasks/LocalAgentTask/LocalAgentTask.tsx` (lines 466-515)

```
export function registerAsyncAgent({
  agentId,
  description,
  prompt,
  selectedAgent,
  setAppState,
  parentAbortController,
  toolUseId,
}): LocalAgentTaskState {
  // Create abort controller (supports hierarchical cancellation)
  const abortController = parentAbortController
    ? createChildAbortController(parentAbortController)
    : createAbortController();

  const taskState: LocalAgentTaskState = {
    ...createTaskStateBase(agentId, 'local_agent', description, toolUseId),
    type: 'local_agent',
    status: 'running',
    agentId,
    prompt,
    selectedAgent,
    agentType: selectedAgent?.agentType ?? 'general-purpose',
    abortController,
    retrieved: false,
    isBackgrounded: true,    // Run in background by default
    pendingMessages: [],
    retain: false,
    diskLoaded: false,
    lastReportedToolCount: 0,
    lastReportedTokenCount: 0,
  };

  // Register to global state
  registerTask(taskState, setAppState);

  return taskState;
}
```

10.3 AbortController Hierarchy

```
Parent AbortController (parent Agent)
    │
    ├── createChildAbortController()
    │         │
    │         ▼
    │   Child AbortController (child Agent)
    │         │
    │         ├── Child Abort → Cancel child agent, not parent
```

```
    │         └── Parent Abort → Automatically cancel all children
    │
    └── Child Agent process IPC communication
          └ <task-notification> event
```

10.4 Progress Tracker

```
// Lines 50-104
export type ProgressTracker = {
  toolUseCount: number;                     // Tool call count
  latestInputTokens: number;               // Latest input token count
  cumulativeOutputTokens: number;          // Cumulative output token count
  recentActivities: ToolActivity[];        // Recent activity records
};

export function updateProgressFromMessage(
  tracker: ProgressTracker,
  message: Message,
  resolveActivityDescription?,
  tools?,
): void {
  // Update latest input token count (including cache read count)
  tracker.latestInputTokens =
    usage.input_tokens +
    (usage.cache_creation_input_tokens ?? 0) +
    (usage.cache_read_input_tokens ?? 0);

  // Accumulate output tokens
  tracker.cumulativeOutputTokens += usage.output_tokens;

  // Track tool calls (skip SYNTHETIC_OUTPUT_TOOL_NAME)
  tracker.toolUseCount += toolUseCount;
}
```

10.5 Task Notification Format

File: lines 252-257

When an agent completes, it notifies the parent agent via `<task-notification>` XML tag:

```
<task-notification>
  <task-id>{taskId}</task-id>
  <status>completed</status>  <!-- completed | failed | killed -->
  <summary>{summary}</summary>
  <result>{finalMessage}</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
</task-notification>
```

Note

The CC system has no structured constraints on sub-agent return report content, which may lead to loss of important information during agent interactions

10.6 Task Output to Disk

```
Agent execution
    │
    ├── Write metadata: ~/.claude/agents/{agentId}/metadata.json
    ├── Write transcript: ~/.claude/agents/{agentId}/transcript.jsonl
    └── Output file path: ~/.claude/agents/{agentId}/output.txt

Parent agent can read output.txt via TaskOutputTool
```

---

11. RemoteAgent and InProcessTeammate

11.1 RemoteAgentTask

File: `tasks/RemoteAgentTask/RemoteAgentTask.tsx`

Remote agents run in the **CCR (Claude Code Remote)** environment:

```typescript
// Startup condition: process.env.CLAUDE_CODE_REMOTE === "1"
async function runRemoteAgent() {
  // 1. Get agent config from environment variables
  const agentConfig: AgentConfig = {
    agentId: process.env.CLAUDE_CODE_AGENT_ID!,
    systemPrompt: process.env.CLAUDE_CODE_SYSTEM_PROMPT!,
    tools: JSON.parse(process.env.CLAUDE_CODE_TOOLS || "[]"),
  };

  // 2. Connect to MCP server
  const mcpClient = await createMCPClient(process.env.MCP_SERVER_URL!);

  // 3. Register MCP tools
  registerMcpTools(mcpClient);

  // 4. Run query loop
  await runQueryWithMCP(agentConfig, mcpClient);
}
```

11.2 InProcessTeammateTask

File: `tasks/InProcessTeammateTask/InProcessTeammateTask.tsx`

In-process Teammate, no subprocess overhead, shared memory:

```typescript
// Features:
// - Runs directly in the main process (no IPC needed)
// - Shares AppStateStore (direct state modification)
// - Cannot spawn other background agents
// - Receives SendMessage via in-memory queue
```

```
interface InProcessTeammateState {
  type: 'in_process_teammate';
  agentId: string;
  teamName: string;
  inbox: MessageQueue;  // In-memory message queue
}
```

Note

- In-process teammates and the parent agent share AppState, file permissions, and tool instances, but each has independent conversation history and identity context (isolated via AsyncLocalStorage)
- Regarding teammate spawning sub-agents:
  - Teammates can spawn normal sub-agents (without name/team_name), but cannot spawn another teammate, and cannot spawn sub-agents with run_in_background: true. Normal sub-agents block the main process (parent agent waits); background sub-agents do not block.
  - Reason: The in-process teammate's lifecycle is controlled by the leader, which runs in the leader's process. **If it spawns a background async sub-agent, that sub-agent's lifecycle management would become chaotic**
  - The system prompt sent to the teammate also explicitly states:

```
- The name, team_name, and mode parameters are not available in this context
- teammates cannot spawn other teammates. Omit them to spawn a subagent.
```

---

12. StreamingToolExecutor - Tool Parallel Orchestration Engine

Note

Benefits of parallel tool invocation:

- Lower latency (saves time)
- Higher token efficiency, fewer turns, less history accumulation (saves money)

12.1 Core Design Principles

File: `services/tools/StreamingToolExecutor.ts`

```
/**
 * Core design for tool parallel orchestration:
 * - Concurrency-safe tools: can execute in parallel with other concurrency-safe tools
 * - Non-concurrency-safe tools: exclusive execution (must run alone)
 * - Results buffered and emitted in original order
 * - Streaming execution: tool results streamed back to API in real time (supports multiple
rounds of tool calls)
 */
export class StreamingToolExecutor { ... }
```

12.2 Core Data Structures

```typescript
type ToolStatus = 'queued' | 'executing' | 'completed' | 'yielded';

type TrackedTool = {
  id: string;                              // Tool call ID
  block: ToolUseBlock;                     // Tool call block from API response
  assistantMessage: AssistantMessage;      // The assistant message this belongs to
  status: ToolStatus;                      // Current status
  isConcurrencySafe: boolean;              // Concurrency safety flag
  promise?: Promise<void>;                 // Execution Promise
  results?: Message[];                     // Final result messages
  pendingProgress: Message[];              // Real-time progress messages (yielded
immediately)
  contextModifiers?: Array<
    (context: ToolUseContext) => ToolUseContext  // Context modifier
  >;
};
```

Note: contextModifier is a patch that tools call back to the execution engine after execution, used to dynamically adjust the runtime environment for subsequent tools (permissions, state, tool pool), not extra messages for the LLM.

	tool.results	contextModifiers
修改对象	对话消息历史（给 LLM 看）	ToolUseContext（给**执行引擎**看）
作用时机	随消息一起发给 API	执行引擎内部消化，不发到 API
效果	LLM 知道工具输出什么	后续工具的执行行为被改变
典型用途	工具返回的文字/错误信息	工具执行后调整权限、状态、工具池

12.3 Concurrency Safety Determination

```typescript
// Lines 104-113
const parsedInput = toolDefinition.inputSchema.safeParse(block.input);
const isConcurrencySafe = parsedInput?.success
  ? (() => {
      try {
        // Call the tool definition's isConcurrencySafe callback
        return Boolean(toolDefinition.isConcurrencySafe(parsedInput.data));
      } catch {
        return false;  // Parse failed → unsafe
      }
    })()
  : false;

// Default safety rules:
// - Read / Glob / Grep → concurrency safe (no side effects)
// - Write / Edit / Bash → not concurrency safe (has side effects)
```

Note

- Does not analyze actual dependencies between tools, only uses a blanket "may produce side effects" check
- Simple and imprecise, but guarantees correctness

- For example, "tool B may write files, not sure what it reads" → take conservative approach: if side effects are possible, always execute serially

12.4 Execution Queue Algorithm

File: lines 140-150

```
private async processQueue(): Promise<void> {
  for (const tool of this.tools) {
    if (tool.status !== 'queued') continue;

    if (this.canExecuteTool(tool.isConcurrencySafe)) {
      // Condition met: start execution
      await this.executeTool(tool);
    } else {
      // Condition not met
      if (!tool.isConcurrencySafe) {
        // Non-concurrency-safe tool encounters conflict: stop scanning (wait for
conflicting tool to complete)
        break;
      }
      // Concurrency-safe tool but slot is full: skip (will process when slot frees up)
    }
  }
}

// Determine if execution is possible
private canExecuteTool(isConcurrencySafe: boolean): boolean {
  const executingNonConcurrent = this.tools.some(
    t => t.status === 'executing' && !t.isConcurrencySafe
  );

  if (executingNonConcurrent) return false;  // An exclusive tool is running
  if (!isConcurrencySafe) return true;        // Non-concurrent tools can always start
(exclusive)
  return true;                               // Concurrency-safe tools can run in parallel
}
```

12.5 Single Tool Execution Flow

```
private async executeTool(tool: TrackedTool): Promise<void> {
  tool.status = 'executing';

  const toolDefinition = findToolByName(this.toolDefinitions, tool.block.name);

  if (!toolDefinition) {
    // Tool doesn't exist: return error
    tool.results = [createUserMessage({
      content: [{
        type: 'tool_result',
        content: `<tool_use_error>Error: No such tool available: ${block.name}
</tool_use_error>`,
```

```
        is_error: true,
        tool_use_id: block.id,
      }],
      toolUseResult: `Error: No such tool available: ${block.name}`,
      sourceToolAssistantUUID: tool.assistantMessage.uuid,
    })];
    tool.status = 'completed';
    return;
  }

  // Permission check
  if (!this.canUseTool(toolDefinition, tool.block.input)) {
    tool.results = [createUserMessage({
      content: [{
        type: 'tool_result',
        content: REJECT_MESSAGE,
        is_error: true,
        tool_use_id: block.id,
      }],
      toolUseResult: REJECT_MESSAGE,
      sourceToolAssistantUUID: tool.assistantMessage.uuid,
    })];
    tool.status = 'completed';
    return;
  }

  // Execute tool (create child AbortController)
  const toolAbortController = createChildAbortController(
    this.siblingAbortController
  );

  // Streaming or normal execution
  if (toolDefinition.stream) {
    // Streaming tool: yield progress in real time
    tool.results = await this.streamTool(
      toolDefinition,
      tool.block.input,
      toolAbortController,
      tool,
    );
  } else {
    // Normal tool: direct call
    tool.results = [await toolDefinition.invoke(
      tool.block.input,
      { ...this.toolUseContext, abortController: toolAbortController }
    )];
  }

  tool.status = 'completed';
}
```

12.6 Error Propagation and Sibling Process Abort

```
// Lines 359-363: When Bash tool errors, abort all sibling processes (multiple tool calls emitted in parallel from the same API response round)
// Reason: Bash errors usually mean the environment is corrupted, sibling process results are not trustworthy

private siblingAbortController: AbortController;  // Child AbortController

// When Bash tool execution errors:
this.hasErrored = true;
this.erroredToolDescription = toolDescription;
// Abort all sibling processes (but not parent query)
this.siblingAbortController.abort('sibling_error');

// Other tools receive abort signal:
if (toolAbortController.signal.aborted) {
  tool.results = [createSyntheticErrorMessage('sibling_error')];
  return;
}
```

Reason: Bash errors usually mean: the environment is corrupted.
The parent is not aborted, giving the main agent decision space: parent query loop continues, main agent receives 3 results, decides to report error, fix, or continue.

12.7 Result Aggregation

```
// getCompletedResults: yield completed results in original order
async function* getRemainingResults(): AsyncGenerator<MessageUpdate> {
  while (true) {
    if (!this.hasPendingResults()) {
      // No pending results: wait (progressAvailableResolve wakes up)
      await new Promise<void>(r => { this.progressAvailableResolve = r; });
    }

    for (const tool of this.tools) {
      if (tool.status === 'completed' && !tool.yielded) {
        // Immediately yield progress messages (real-time)
        for (const msg of tool.pendingProgress) {
          yield { message: msg };
        }

        // Yield final result
        for (const msg of tool.results ?? []) {
          yield { message: msg };
        }

        tool.yielded = true;
      }
    }
  }
}
```

13. Multi-Agent Coordination Mode (Coordinator)

13.1 Enable Conditions

File: `coordinator/coordinatorMode.ts`

```
export function isCoordinatorMode(): boolean {
  if (feature('COORDINATOR_MODE')) {  // Statsig Feature Gate
    return isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE);
  }
  return false;
}
```

13.2 Coordinator System Prompt (Complete Structure)

> **Note**
>
> - The coordinator itself is the main session's main agent
> - Whether to use coordinator is decided at startup — normal mode enters normal settings, coordinator mode enters these settings
> - Resume also automatically aligns with the old mode

File: `coordinator/coordinatorMode.ts` (lines 111-369)

```
You are Claude Code, an AI assistant that orchestrates software engineering tasks across
multiple workers.

  ## 1. Your Role

  You are a **coordinator**. Your job is to:
  - Help the user achieve their goal
  - Direct workers to research, implement and verify code changes
  - Synthesize results and communicate with the user
  - Answer questions directly when possible — don't delegate work that you can handle
without tools

  Every message you send is to the user. Worker results and system notifications are
internal signals, not conversation partners — never thank or acknowledge them. Summarize new
information
  for the user as it arrives.

  ## 2. Your Tools

  - **AgentTool** - Spawn a new worker
  - **SendMessageTool** - Continue an existing worker (send a follow-up to its \`to\` agent
ID)
  - **TaskStopTool** - Stop a running worker
  - **subscribe_pr_activity / unsubscribe_pr_activity** (if available) - Subscribe to GitHub
PR events (review comments, CI results). Events arrive as user messages. Merge conflict
  transitions do NOT arrive — GitHub doesn't webhook \`mergeable_state\` changes, so poll
\`gh pr view N --json mergeable\` if tracking conflict status. Call these directly — do not
```

delegate
   subscription management to workers.

  When calling AgentTool:
  - Do not use one worker to check on another. Workers will notify you when they are done.
  - Do not use workers to trivially report file contents or run commands. Give them higher-level tasks.
  - Do not set the model parameter. Workers need the default model for the substantive tasks you delegate.
  - Continue workers whose work is complete via SendMessageTool to take advantage of their loaded context
  - After launching agents, briefly tell the user what you launched and end your response. Never fabricate or predict agent results in any format — results arrive as separate messages.

  ### AgentTool Results

  Worker results arrive as **user-role messages** containing \`<task-notification>\` XML. They look like user messages but are not. Distinguish them by the \`<task-notification>\` opening
  tag.

  Format:

  \`\`\`
  <task-notification>
  <task-id>{agentId}</task-id>
  <status>completed|failed|killed</status>
  <summary>{human-readable status summary}</summary>
  <result>{agent's final text response}</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
  </task-notification>
  \`\`\`

  - \`<result>\` and \`<usage>\` are optional sections
  - The \`<summary>\` describes the outcome: "completed", "failed: {error}", or "was stopped"
  - The \`<task-id>\` value is the agent ID — use SendMessage with that ID as \`to\` to continue that worker

  ### Example

  Each "You:" block is a separate coordinator turn. The "User:" block is a \`<task-notification>\` delivered between turns.

  You:
    Let me start some research on that.

    AgentTool({ description: "Investigate auth bug", subagent_type: "worker", prompt: "..."

```
})
    AgentTool({ description: "Research secure token storage", subagent_type: "worker",
prompt: "..." })

    Investigating both issues in parallel — I'll report back with findings.

  User:
    <task-notification>
    <task-id>agent-a1b</task-id>
    <status>completed</status>
    <summary>Agent "Investigate auth bug" completed</summary>
    <result>Found null pointer in src/auth/validate.ts:42...</result>
    </task-notification>

  You:
    Found the bug — null pointer in confirmTokenExists in validate.ts. I'll fix it.
    Still waiting on the token storage research.

    SendMessageTool({ to: "agent-a1b", message: "Fix the null pointer in
src/auth/validate.ts:42..." })

  ## 3. Workers

  When calling AgentTool, use subagent_type \`worker\`. Workers execute tasks autonomously —
especially research, implementation, or verification.

  ${workerCapabilities}

  ## 4. Task Workflow

  Most tasks can be broken down into the following phases:

  ### Phases

  | Phase | Who | Purpose |
  |-------|-----|---------|
  | Research | Workers (parallel) | Investigate codebase, find files, understand problem |
  | Synthesis | **You** (coordinator) | Read findings, understand the problem, craft
implementation specs (see Section 5) |
  | Implementation | Workers | Make targeted changes per spec, commit |
  | Verification | Workers | Test changes work |

  ### Concurrency

  **Parallelism is your superpower. Workers are async. Launch independent workers
concurrently whenever possible — don't serialize work that can run simultaneously and look
for opportunities
   to fan out. When doing research, cover multiple angles. To launch workers in parallel,
make multiple tool calls in a single message.**

  Manage concurrency:
  - **Read-only tasks** (research) — run in parallel freely
  - **Write-heavy tasks** (implementation) — one at a time per set of files
```

- **Verification** can sometimes run alongside implementation on different file areas

What Real Verification Looks Like

Verification means **proving the code works**, not confirming it exists. A verifier that rubber-stamps weak work undermines everything.

- Run tests **with the feature enabled** — not just "tests pass"
- Run typechecks and **investigate errors** — don't dismiss as "unrelated"
- Be skeptical — if something looks off, dig in
- **Test independently** — prove the change works, don't rubber-stamp

Handling Worker Failures

When a worker reports failure (tests failed, build errors, file not found):
- Continue the same worker with SendMessageTool — it has the full error context
- If a correction attempt fails, try a different approach or report to the user

Stopping Workers

Use TaskStopTool to stop a worker you sent in the wrong direction — for example, when you realize mid-flight that the approach is wrong, or the user changes requirements after you launched
the worker. Pass the \`task_id\` from the AgentTool tool's launch result. Stopped workers can be continued with SendMessageTool.

5. Writing Worker Prompts

Workers can't see your conversation. Every prompt must be self-contained with everything the worker needs. After research completes, you always do two things: (1) synthesize findings
into a specific prompt, and (2) choose whether to continue that worker via SendMessageTool or spawn a fresh one.

Always synthesize — your most important job

When workers report research findings, **you must understand them before directing follow-up work**. Read the findings. Identify the approach. Then write a prompt that proves you understood by including specific file paths, line numbers, and exactly what to change.

Never write "based on your findings" or "based on the research." These phrases delegate understanding to the worker instead of doing it yourself. You never hand off understanding to
another worker.

Continue vs. spawn by context overlap

Situation	Mechanism	Why
Research explored exactly the files that need editing	**Continue** (SendMessageTool)	Worker already has the files in context AND now gets a clear plan
Research was broad but implementation is narrow	**Spawn fresh** (AgentTool)	Avoid dragging along exploration noise; focused context is cleaner

Correcting a failure or extending recent work	**Continue**	Worker has the error context and knows what it just tried
Verifying code a different worker just wrote	**Spawn fresh**	Verifier should see the code with fresh eyes
First implementation attempt used wrong approach	**Spawn fresh**	Wrong-approach context pollutes the retry; clean slate avoids anchoring on failed path
Completely unrelated task	**Spawn fresh**	No useful context to reuse

Prompt tips

Good examples:

1. Implementation: "Fix the null pointer in src/auth/validate.ts:42. The user field can be undefined when the session expires. Add a null check and return early with an appropriate error.
Commit and report the hash."

2. Precise git operation: "Create a new branch from main called 'fix/session-expiry'. Cherry-pick only commit abc123 onto it. Push and create a draft PR targeting main. Add
anthropics/claude-code as reviewer. Report the PR URL."

3. Correction (continued worker, short): "The tests failed on the null check you added — validate.test.ts:58 expects 'Invalid session' but you changed it to 'Session expired'. Fix the
assertion. Commit and report the hash."

Bad examples:

4. "Fix the bug we discussed" — no context, workers can't see your conversation
5. "Based on your findings, implement the fix" — lazy delegation
6. "Create a PR for the recent changes" — ambiguous scope
7. "Something went wrong with the tests, can you look?" — no error, no path, no direction

6. Example Session

User: "There's a null pointer in the auth module. Can you fix it?"

You:
Let me investigate first.

AgentTool({ description: "Investigate auth bug", subagent_type: "worker", prompt: "Investigate the auth module in src/auth/. Find where null pointer exceptions could occur around session
handling and token validation... Report specific file paths, line numbers, and types involved. Do not modify files." })
AgentTool({ description: "Research auth tests", subagent_type: "worker", prompt: "Find all test files related to src/auth/. Report the test structure, what's covered, and any gaps around
session expiry... Do not modify files." })

Investigating from two angles — I'll report back with findings.

User:

```
    <task-notification>
    <task-id>agent-a1b</task-id>
    <status>completed</status>
    <summary>Agent "Investigate auth bug" completed</summary>
    <result>Found null pointer in src/auth/validate.ts:42...</result>
    </task-notification>

  You:
    Found the bug — null pointer in validate.ts:42.
    SendMessageTool({ to: "agent-a1b", message: "Fix the null pointer in
src/auth/validate.ts:42. Add a null check before accessing user.id — if null, return 401
with 'Session expired'.
  Commit and report the hash." })

    Fix is in progress.
```

The system prompt contains the following sections:

1. **Role definition**: Coordinator, managing multiple software engineering Workers
2. **Available tools**:
   - `Agent` - Spawn new Worker
   - `SendMessage` - Continue existing Worker
   - `TaskStop` - Stop Worker
   - `subscribe_pr_activity` / `unsubscribe_pr_activity` - GitHub PR event subscription
3. **Worker capabilities**:
   ```
   // Simple mode (CLAUDE_CODE_SIMPLE=1):
   // Workers have access to Bash, Read, and Edit tools, plus MCP tools...

   // Full mode:
   // Workers have access to standard tools, MCP tools,
   // and project skills via the Skill tool...
   ```
4. **Parallelism strategy**:
   - Research tasks: free parallelism (read-only)
   - Implementation tasks: one Worker at a time per file group
5. **Communication protocol**: Receive Worker results via `<task-notification>` XML tag
6. **Continue vs. Spawn decision matrix**:
   - High context overlap → `SendMessage` to continue
   - Low context overlap / wrong approach / unrelated task → Spawn new Worker
7. **Workflow**:
   - Research: Workers (parallel)
   - Synthesis: Coordinator
   - Implementation: Workers
   - Verification: Workers

Comparison of the two mode prompts:

1. Role definition

Normal mode: General-purpose agent, completes all tasks directly, calls Read / Write / Bash tools directly.
Coordinator mode: Coordinator role, does not execute directly, only dispatches. All file operations are prohibited, all communication with workers via AgentTool / SendMessageTool / TaskStopTool, main agent itself does not touch project files.

2. Tool layer

Normal mode has the complete tool pool (Read/Write/Bash/Grep/WebFetch, etc.), tool calls execute directly.
Coordinator mode has extremely limited tool pool — only three tools: AgentTool, SendMessageTool, TaskStopTool. All tool calls are delegated to worker agents, main agent does not touch project files.

3. Concurrency model

Normal mode is tool-level parallelism, StreamingToolExecutor automatically schedules multiple tools to execute in parallel (Read/Glob/Grep can be parallel, Bash/Write/Edit are serial).
Coordinator mode is task-level parallelism, main agent proactively designs parallelism strategy: read tasks (research) are parallel, write tasks (implementation) are serial. Core principle: "Parallelism is your superpower. Workers are async. Launch independent workers concurrently."

4. Task lifecycle

Normal mode is linear execution: user request → main agent completes directly → reports result.
Coordinator mode is a four-phase pipeline:

- Phase 1 Research: Workers research codebase in parallel, understand the problem
- Phase 2 Synthesis: Coordinator digests research results, understands the problem, writes precise implementation specs (with file paths, line numbers)
- Phase 3 Implementation: Workers implement according to specs, commit code
- Phase 4 Verification: Workers independently and adversarially verify

5. Communication model

Normal mode messages come directly from users, main agent responds directly.
Coordinator mode most messages the main agent receives are `<task-notification>` (worker results) rather than real user input. Worker results and system notifications are internal signals, not conversation partners — never thank or acknowledge them.

6. Verification approach

Normal mode main agent can verify by itself + optional verification subagent, allows self-assign PASS/FAIL.
Coordinator mode verification is done by independent workers, only the verifier assigns a verdict, main agent is prohibited from self-assigning. Verification means proving the code works, not confirming it exists.

7. Anti-pattern constraints

Coordinator prompt specifically lists delegation bad smells to avoid:

- "Fix the bug we discussed" — workers can't see conversation, no context
- "Based on your findings..." — delegates understanding responsibility to worker, doesn't do it yourself
- "Create a PR for the recent changes" — ambiguous: which changes? Which branch?
- "Something went wrong, can you look?" — no error info, no path, no direction
  Normal mode doesn't have this dedicated delegation anti-pattern list.

8. Error recovery strategy

Normal mode retries, changes approach, or asks user on failure.
Coordinator mode continues the same worker (SendMessageTool) on failure, because it has complete error context. If a correction attempt fails, try a different approach or report to the user.

13.3 Coordinator Tool Context

```
// getCoordinatorUserContext (starting at line 80)
export function getCoordinatorUserContext(
  mcpClients: ReadonlyArray<{ name: string }>,
  scratchpadDir?: string,  // Statsig Feature Gate: tengu_scratch
): { [k: string]: string } {
  // Build workerToolsContext
  const workerTools = isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)
    ? [BASH_TOOL_NAME, FILE_READ_TOOL_NAME, FILE_EDIT_TOOL_NAME].sort().join(', ')
    : Array.from(ASYNC_AGENT_ALLOWED_TOOLS)
        .filter(name => !INTERNAL_WORKER_TOOLS.has(name))  // Exclude internal tools
        .sort().join(', ');

  return { workerToolsContext: workerToolsPrompt };
}
```

13.4 Worker Tool Whitelist

File: `constants/tools.ts`

```
// Tools available to Coordinator Workers (ASYNC_AGENT_ALLOWED_TOOLS)
// Used internally by AgentTool: only these tools are allowed to pass through AgentTool
spawn
export const ASYNC_AGENT_ALLOWED_TOOLS: ReadonlySet<string> = new Set([
    FILE_READ_TOOL_NAME,        // Read
    WEB_SEARCH_TOOL_NAME,      // WebSearch
    TODO_WRITE_TOOL_NAME,       // TodoWrite
    GREP_TOOL_NAME,             // Grep
    WEB_FETCH_TOOL_NAME,        // WebFetch
    GLOB_TOOL_NAME,             // Glob
    ...SHELL_TOOL_NAMES,        // Bash, PowerShell
    FILE_EDIT_TOOL_NAME,        // Edit
    FILE_WRITE_TOOL_NAME,       // Write
    NOTEBOOK_EDIT_TOOL_NAME,    // NotebookEdit
    SKILL_TOOL_NAME,            // Skill
    SYNTHETIC_OUTPUT_TOOL_NAME,  // SyntheticOutput
    TOOL_SEARCH_TOOL_NAME,      // ToolSearch
    ENTER_WORKTREE_TOOL_NAME,    // EnterWorktree
    EXIT_WORKTREE_TOOL_NAME,     // ExitWorktree
]);
```

IN_PROCESS_TEAMMATE_ALLOWED_TOOLS (additionally available to in-process teammates)

```
export const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
    TASK_CREATE_TOOL_NAME,   // TaskCreate
    TASK_GET_TOOL_NAME,      // TaskGet
```

```
    TASK_LIST_TOOL_NAME,     // TaskList
    TASK_UPDATE_TOOL_NAME,   // TaskUpdate
    SEND_MESSAGE_TOOL_NAME,  // SendMessage
    // Conditionally enabled (AGENT_TRIGGERS feature gate):
    ...(feature('AGENT_TRIGGERS')
      ? [CRON_CREATE_TOOL_NAME, CRON_DELETE_TOOL_NAME, CRON_LIST_TOOL_NAME]
      : []),
  ])
```

13.5 Internal Tools (Invisible to Workers)

```
// Coordinator-specific tools, invisible to Workers
const INTERNAL_WORKER_TOOLS = new Set([
  TEAM_CREATE_TOOL_NAME,    // TeamCreateTool
  TEAM_DELETE_TOOL_NAME,    // TeamDeleteTool
  SEND_MESSAGE_TOOL_NAME,   // SendMessageTool
  SYNTHETIC_OUTPUT_TOOL_NAME, // SyntheticOutputTool
]);
```

---

14. Context Compaction System

> **Note**
>
> See the corresponding Context Management document for details

14.1 Compaction Strategy System

Claude Code implements a **three-level compaction system**:

```
Token budget
    │
    ├── Normal zone (< AUTOCOMPACT_THRESHOLD)
    │     └─ No compaction, normal operation
    │
    ├── Warning zone (> WARNING_THRESHOLD)
    │     └─ Display warning prompt
    │
    ├── Auto Compact trigger point (> AUTOCOMPACT_THRESHOLD)
    │     └─ Automatically trigger compaction
    │
    └── Manual compaction zone (/compact command)
          ├─ snip mode: truncate only
          ├─ full mode: full compaction
          └─ aggressive mode: aggressive summarization
```

14.2 Precise Thresholds

File: `services/compact/autoCompact.ts`

```
// Maximum tokens for compaction summary output
// Based on observation that p99.99 compaction summary output is 17,387 tokens
```

```
const MAX_OUTPUT_TOKENS_FOR_SUMMARY = 20_000;

// Auto Compact trigger threshold buffer
export const AUTOCOMPACT_BUFFER_TOKENS = 13_000;

// Warning threshold buffer
export const WARNING_THRESHOLD_BUFFER_TOKENS = 20_000;

// Error threshold buffer
export const ERROR_THRESHOLD_BUFFER_TOKENS = 20_000;

// Manual compaction buffer
export const MANUAL_COMPACT_BUFFER_TOKENS = 3_000;

// Auto compaction failure circuit breaker threshold
const MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3;
// Note: 2026-03-10 data shows ~250K API calls wasted per day due to consecutive compaction
failures
```

14.3 Auto Compaction Threshold Calculation

```
// Lines 72-91
export function getAutoCompactThreshold(model: string): number {
  // Effective context window = model context window - reserved tokens for summary output
  const effectiveContextWindow = getEffectiveContextWindowSize(model);

  let autocompactThreshold =
    effectiveContextWindow - AUTOCOMPACT_BUFFER_TOKENS;
  // = effectiveContextWindow - 13,000

  // Environment variable override (for testing)
  const envPercent = process.env.CLAUDE_AUTOCOMPACT_PCT_OVERRIDE;
  if (envPercent) {
    const parsed = parseFloat(envPercent);
    if (!isNaN(parsed) && parsed > 0 && parsed <= 100) {
      return Math.floor(effectiveContextWindow * (parsed / 100));
    }
  }

  return autocompactThreshold;
}

// effectiveContextWindow calculation (lines 33-49)
export function getEffectiveContextWindowSize(model: string): number {
  const reservedTokensForSummary = Math.min(
    getMaxOutputTokensForModel(model),
    MAX_OUTPUT_TOKENS_FOR_SUMMARY,  // 20,000
  );

  let contextWindow = getContextWindowForModel(model, getSdkBetas());

  // Environment variable override
  if (process.env.CLAUDE_CODE_AUTO_COMPACT_WINDOW) {
```

```
    const parsed = parseInt(process.env.CLAUDE_CODE_AUTO_COMPACT_WINDOW, 10);
    if (!isNaN(parsed) && parsed > 0) {
      contextWindow = Math.min(contextWindow, parsed);
    }
  }

  return contextWindow - reservedTokensForSummary;
}
```

14.4 Compaction Execution Algorithm

File: `services/compact/compact.ts`

```
export async function compactConversation(
  messages: Message[],
  options: CompactOptions,
): Promise<CompactionResult> {
  // 1. Identify compactable start point (pivot index)
  const pivotIndex = findPivotIndex(messages);

  // 2. Generate summary for messages before pivot
  const summaryPrompt = buildSummaryPrompt(messages.slice(0, pivotIndex));
  const summaryMessage = await callSummaryModel(summaryPrompt);

  // 3. Build compacted message list
  const compressedMessages = [
    ...messages.slice(0, pivotIndex).map(m => markAsCompacted(m)),
    createSummaryMessage(summaryMessage),
    ...messages.slice(pivotIndex),
  ];

  // 4. Restore recent attachment files (up to 5, total 5,000 tokens)
  const restoredFiles = restoreRecentAttachments(
    compressedMessages,
    { maxFiles: 5, maxTokensPerFile: 5_000 }
  );

  // 5. Restore skill information (5,000 tokens/skill, total 25,000 tokens)
  const restoredSkills = restoreSkillInfo(
    compressedMessages,
    { maxSkills: 5, maxTokensPerSkill: 5_000 }
  );

  return {
    messages: compressedMessages,
    restoredFiles,
    restoredSkills,
    pivotIndex,
    summaryMessage,
  };
}
```

14.5 Tombstone Mechanism After Compaction

```
// Messages after compaction are marked as tombstones (TombstoneMessage)
// Displayed as gray placeholders, not counted in tokens
interface TombstoneMessage {
  type: 'tombstone';
  originalCount: number;  // Number of messages compacted
  summaryId: string;      // Summary ID (can be used for expansion)
}

// Compaction boundary message
interface CompactBoundaryMessage {
  type: 'compact_boundary';
  pivotUuid: string;
  messageCountBefore: number;
  tokensSaved: number;
}
```

14.6 Compaction Circuit Breaker

```
// Stop retrying after 3 consecutive compaction failures
// Prevents wasted API calls from irrecoverable context overflow

export async function autoCompactIfNeeded(
  messages: Message[],
  tracking?: AutoCompactTrackingState,
): Promise<{ wasCompacted: boolean; consecutiveFailures?: number }> {
  // Check if threshold exceeded
  const currentTokens = tokenCountWithEstimation(messages);
  const threshold = getAutoCompactThreshold(currentModel);

  if (currentTokens <= threshold) {
    return { wasCompacted: false };
  }

  try {
    // Attempt compaction
    const result = await compactConversation(messages, options);
    return { wasCompacted: true, compactionResult: result };
  } catch (error) {
    const consecutiveFailures = (tracking?.consecutiveFailures ?? 0) + 1;

    if (consecutiveFailures >= MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES) {
      // Circuit breaker: stop retrying
      logError(`Auto-compact failed ${consecutiveFailures} times, circuit breaker
triggered`);
      return { wasCompacted: false, consecutiveFailures };
    }

    // Continue retrying
    return { wasCompacted: false, consecutiveFailures };
  }
}
```

14.7 Compaction-triggered Transcript Flushing

```
// In query.ts, flush in-memory transcript before compaction boundary
// Lines 701-715
if (compactBoundaryIndex !== undefined) {
  // Before writing compact boundary message:
  // 1. Flush in-memory messages to transcript
  const tailIdx = findTailIndex(state.messages, compactBoundaryIndex);
  await recordTranscript(state.messages.slice(0, tailIdx + 1));
}
```

---

15. Memory System

> **Note**
>
> See the corresponding Memory System document for details

15.1 Memory Types

File: `memdir/memdir.ts`

```
export const MEMORY_TYPES = ['user', 'feedback', 'project', 'reference'] as const;
export type MemoryType = (typeof MEMORY_TYPES)[number];
```

Type	Content	When to save
`user`	User role, goals, preferences	When learning about user's background
`feedback`	User guidance (things to avoid / things to persist)	When user corrects
`project`	Project state, goals, known bugs	When project context changes
`reference`	External system references (URLs, document locations)	When discovering external information

15.2 Memory Directory Structure

```
~/.claude/projects/{slug}/memory/
    │
    ├── MEMORY.md         # Index file (max 200 lines, 25KB)
    │                     # Format: `- [Title](file.md) — one-line hook`
    │
    ├── user/
    │     └── user_role.md    # User information
    │
    ├── feedback/
    │     └── feedback_*.md   # Feedback records
    │
    ├── project/
    │     └── project_*.md    # Project context
    │
    └── reference/
          └── reference_*.md  # External references
```

15.3 Memory File Format

```
---
name: {{memory name}}
description: {{one-line description — used to decide relevance}}
type: {{user | feedback | project | reference}}
---

{{memory content — feedback/project types need to include WHY and How to apply}}
```

Special format for feedback type:

```
---
name: avoid_synthetic_mocks
description: Don't mock the database, use real integration tests
type: feedback
---

Don't use mocked databases in integration tests — **Why:** Last quarter mocked tests passed
but the production migration failed. **How to apply:** All tests must hit a real database,
using testcontainers or in-memory databases.
```

15.4 Memory Loading Flow

```
// memdir.ts
export async function loadMemoryPrompt(): Promise<string | null> {
  // 1. Read MEMORY.md index file
  const memoryIndex = await readFile(memoryIndexPath, 'utf-8');

  // 2. Parse index lines
  const entries = parseMemoryIndex(memoryIndex);

  // 3. Filter relevant entries (based on current query)
  const relevantEntries = filterRelevantEntries(entries, currentQuery);

  // 4. Load relevant memory file contents
  const memoryContents = await Promise.all(
    relevantEntries.map(entry => readMemoryFile(entry.path))
  );

  // 5. Inject into system prompt
  return buildMemoryInjectionPrompt(memoryContents);
}
```

15.5 Relationship Between Memory and System Prompt

File: `QueryEngine.ts` (lines 286-325)

```
// Memory injected as dynamic part of system prompt
async function fetchSystemPromptParts() {
  const [defaultSystemPrompt, userContext, systemContext] = await Promise.all([
    getDefaultSystemPrompt(),           // Static part
```

```
    getUserContext(),                   // User information
    getSystemContext(),                 // System information
  ]);

  return {
    defaultSystemPrompt,
    userContext,
    systemContext,
  };
}

// MEMORY_AUTO_MEM_PATH: Auto memory path
// User can specify via CLAUDE_CODE_AUTO_MEM_PATH environment variable
// Custom prompt can override this path
```

15.6 Team Memory

File: `memdir/teamMemPrompts.ts`

```
// Shared memory in Team mode
interface TeamMemory {
  teamId: string;
  sharedContext: MemoryEntry[];   // Team-shared context
  individualContext: {             // Per-member context
    [teammateId: string]: MemoryEntry[];
  };
}

// Team memory characteristics:
// - Shared layer: visible to all members (project goals, architecture decisions)
// - Individual layer: private to each member (their respective current tasks)
```

---

16. Permissions System

16.1 Permission Types

```
type Permission =
  | 'bash'              // Shell command execution
  | 'bash-sudo'         // Privileged shell commands
  | 'edit'              // File editing
  | 'write'             // File writing
  | 'read'              // File reading
  | 'network'           // Network requests
  | 'clipboard'         // Clipboard access
  | 'run-agent'         // Launch sub-agent
  | 'use-mcp'           // Use MCP servers
  | 'web-search'        // Internet search
  | 'web-fetch';        // Web page fetching
```

16.2 Permission Levels

```
enum PermissionLevel {
  DENY = 0,     // Explicitly denied
  PROMPT = 1,   // Ask every time
  ALLOW = 2,    // Allowed
  TRUST = 3,    // Fully trusted
}

// Permission rule matching
interface PermissionRule {
  pattern: string | RegExp;  // Command/file path matching pattern
  level: PermissionLevel;
}
```

16.3 Permission Check Flow

File: `hooks/useCanUseTool.ts`

```
type CanUseToolFn = (
  tool: Tool,
  toolInput: unknown,
) => PermissionCheckResult;

interface PermissionCheckResult {
  allowed: boolean;
  reason?: string;
  behavior: 'allow' | 'deny' | 'prompt';
}

// Check result behavior
function checkPermission(
  permission: Permission,
  tool: Tool,
  toolInput: unknown,
): PermissionCheckResult {
  // 1. Get configured permission level
  const level = getPermissionLevel(permission);

  switch (level) {
    case DENY:
      return { allowed: false, reason: 'denied_by_config', behavior: 'deny' };

    case TRUST:
      return { allowed: true, behavior: 'allow' };

    case ALLOW:
      return { allowed: true, behavior: 'allow' };

    case PROMPT:
      // Show dialog to ask user
      if (tool.allowedWhenPaused) {
        return { allowed: true, behavior: 'allow' };
      }
      return { allowed: false, reason: 'requires_user_confirmation', behavior: 'prompt' };
```

```
  }
}
```

16.4 Agent Permission Modes

```
// AgentTool supports custom permission modes
type PermissionMode =
  | 'acceptEdits'     // Accept edits (default)
  | 'restricted'      // Restricted (read-only tools)
  | 'browseOnly'      // Browse only (no writes)
  | 'custom';        // Custom rules

// Agent definitions can specify permission mode
interface AgentDefinition {
  permissionMode?: PermissionMode;
}
```

16.5 Permission Denial Tracking

```
// Permission tracking in QueryEngine.ts
// Used to report permission usage to SDK callers
class QueryEngine {
  private permissionDenials: SDKPermissionDenial[] = [];

  private wrappedCanUseTool(
    originalCanUseTool: CanUseToolFn,
  ): CanUseToolFn {
    return (tool, input) => {
      const result = originalCanUseTool(tool, input);
      if (result.behavior !== 'allow') {
        // Track denial/prompt events
        this.permissionDenials.push({
          toolName: tool.name,
          reason: result.reason ?? 'permission_check_failed',
          timestamp: Date.now(),
        });
      }
      return result;
    };
  }
}
```

---

17. State Management (AppStateStore)

17.1 Complete State Structure

File: `state/AppStateStore.ts`

```
export type AppState = DeepImmutable<{
  // ===== Configuration =====
  settings: SettingsJson;              // User configuration
  verbose: boolean;                  // Verbose mode
```

```
mainLoopModel: ModelSetting;       // Main loop model
toolPermissionContext: ToolPermissionContext; // Tool permission context

// ===== MCP =====
mcp: {
  clients: MCPServerConnection[];    // Connected MCP servers
  tools: Tool[];                     // MCP tools (dynamically registered)
  commands: Command[];               // MCP commands
  resources: Record<string, ServerResource[]>; // MCP resources
  pluginReconnectKey: number;        // Reconnect key
};

// ===== Task Management =====
tasks: { [taskId: string]: TaskState };  // All agent tasks
agentNameRegistry: Map<string, AgentId>;  // Name → AgentId routing
foregroundedTaskId?: string;             // Foreground task
viewingAgentTaskId?: string;             // Agent being viewed

// ===== File History =====
fileHistory: FileHistoryState;

// ===== Task List =====
todos: { [agentId: string]: TodoList };

// ===== Notifications =====
notifications: {
  current: Notification | null;      // Current notification
  queue: Notification[];             // Notification queue
};

// ===== Thinking Mode =====
thinkingEnabled: boolean | undefined;

// ===== Session Hooks =====
sessionHooks: SessionHooksState;

// ===== Team Context =====
teamContext?: {
  teamName: string;
  teamFilePath: string;
  leadAgentId: string;
  selfAgentId?: string;
  teammates: { [teammateId: string]: TeammateState };
};

// ===== Message Inbox =====
inbox: { messages: Array<InboxMessage> };

// ===== Speculation/Prefill State =====
speculation: SpeculationState;

// ===== Coordinator =====
```

```
  coordinatorTaskIndex: number;
}>;
```

17.2 State Update Patterns

```
// Use functional updates, avoid direct mutation
interface SetAppStateFn {
  (setter: (prev: AppState) => AppState): void;
}

// Example: Register agent
function registerAsyncAgent(params, setAppState: SetAppStateFn) {
  setAppState(prev => ({
    ...prev,
    tasks: {
      ...prev.tasks,
      [taskId]: createTaskState(taskId, params),
    },
  }));
}

// Example: Register agent name routing
setAppState(prev => {
  const next = new Map(prev.agentNameRegistry);
  next.set(name, agentId);
  return { ...prev, agentNameRegistry: next };
});
```

17.3 Team Context State

```
// In Team mode, teamContext stores team state
interface TeamContext {
  teamName: string;          // Team name
  teamFilePath: string;      // Team definition file path
  leadAgentId: string;       // Lead agent (Coordinator)
  selfAgentId?: string;      // Current agent's ID
  teammates: {
    [teammateId: string]: {
      agentId: string;
      status: TaskStatus;
      inbox: Message[];
      lastSeen: number;
    };
  };
}
```

---

18. Slash Command System

> **Note**
>
> - Slash commands are a special input format for triggering **predefined workflows**

- Users can trigger by typing `/command-name [arguments]` in the input box, e.g., `/resume`, `/config`
- **Regular users can define their own slash commands** by writing a markdown file in the .claude/skills/ directory

18.1 Command Registration

File: `commands/` (88+ commands)

```
interface Command {
  name: string;               // Command name (without /)
  description: string;         // Help description
  aliases?: string[];          // Aliases
  flags?: CommandFlag[];      // Supported arguments
  handler: CommandHandler;    // Handler function (supports async generator)
}

// Command priority (for race condition handling)
type CommandPriority =
  | 'pause_after_next'
  | 'pause_immediately'
  | 'resume'
  | 'cancel_resume';
```

18.2 Main Command List

Command	File	Function
`/commit`	`commands/commit/`	Generate git commit
`/review`	`commands/review/`	Code review
`/config`	`commands/config/`	Configuration management
`/init`	`commands/init/`	Project initialization
`/help`	`commands/help/`	Help information
`/compact`	`commands/compact/`	Context compaction
`/model`	`commands/model/`	Switch model
`/retry`	`commands/retry/`	Retry
`/undo`	`commands/undo/`	Undo
`/skill`	`commands/skills/`	Skill management
`/agent`	`commands/agents/`	Agent management
`/exit`	`commands/exit/`	Exit
`/prompt`	`commands/prompt/`	Custom prompts
`/attach`	`commands/attach/`	Attachment handling

18.3 Command Execution Flow

```
User enters /compact --snip
    │
    ▼
InputParser.parse(input)
    │  Parsed as { command: 'compact', args: ['--snip'] }
```

```
    ▼
CommandRegistry.get('compact')
    │  Get command definition and priority
    ▼
checkPriorityConflict()  // Check priority conflict with pending commands
    │
    ▼
CommandLifecycleNotifier.notify('starting')
    │  Notify lifecycle event
    ▼
executeCommand(handler, args, flags)
    │
    ├── Pause current queryLoop
    ├── Execute command handler
    ├── Command may yield messages to query
    └── After command completes, resume or terminate query
    │
    ▼
CommandLifecycleNotifier.notify('completed')
```

---

19. API Call Layer

19.1 Claude API Client

File: `services/api/claude.ts`

```
// API version
const ANTHROPIC_VERSION = '2023-06-01';

// Core request construction
interface ClaudeAPIParams {
  model: string;
  system: string;           // Rendered system prompt
  messages: APIMessage[];  // Formatted messages
  max_tokens: number;
  temperature?: number;
  tools?: APIToolDefinition[];
  // Beta features
  betas?: string[];
  // Prompt caching
  cache_control?: CacheControl;
}

// Streaming response handling
async function* streamClaudeResponse(
  response: Response,
): AsyncGenerator<StreamEvent | AssistantMessage> {
  const reader = response.body?.getReader();
  const decoder = new TextDecoder();

  for await (const chunk of streamReader(reader, decoder)) {
    yield* parseSSEEvent(chunk);  // Parse Server-Sent Events
```

```
  }
}
```

19.2 Model Configuration

File: `utils/model/model.ts`

```
export function getRuntimeMainLoopModel(): string {
  // 1. Read model setting from settings.json
  // 2. Check CLAUDE_CODE_MODEL environment variable
  // 3. Check --model command-line argument
  // 4. Fall back to default model (sonnet-4-5)
}

export function getDefaultSubagentModel(): string {
  // Sub-agent default model (may differ from main model)
  // Usually faster/cheaper than the main model
  return 'sonnet-4-5';
}
```

19.3 Token Budget Management

File: `query/tokenBudget.ts`

```
// Enabled based on feature('TOKEN_BUDGET')
export function createBudgetTracker() {
  // Track token consumption per turn
  // Trigger auto-continue when accumulated consumption reaches threshold
  // Supports +500,000 token auto-continue (beta feature: task-budgets-2026-03-13)
}

export function checkTokenBudget(
  usage: NonNullableUsage,
  budgetTracker: BudgetTracker,
): 'continue' | 'stop';
```

---

20. Key Algorithm Flow Diagrams

20.1 Complete Query Loop Flow

> **Note**
>
> Note that the context compaction check is performed earliest, right when entering the query loop

```
┌──────────────────────────────────────────────────────────────────────────┐
│                         queryLoop() Start                                │
│                                                                          │
│  State = {                                                               │
│    messages: params.messages,                                            │
│    toolUseContext: params.toolUseContext,                                │
│    autoCompactTracking: undefined,                                       │
│    maxOutputTokensRecoveryCount: 0,                                      │
```

```
│    hasAttemptedReactiveCompact: false,                              │
│    turnCount: 1,                                                    │
│    ...                                                              │
│  }                                                                  │
│                                                                     │
│  budgetTracker = createBudgetTracker()  // If TOKEN_BUDGET enabled│
│  taskBudgetRemaining = undefined                                    │
│  config = buildQueryConfig()  // Snapshot immutable config          │
└─────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────┐
│                  while (true) — Main Loop                           │
│                                                                     │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ ITERATION START: turnCount = N                                │   │
│  │                                                               │   │
│  │ 1. compactIfNeeded()                                          │   │
│  │    ├─ tokenCountWithEstimation(messages)                      │   │
│  │    ├─ If > AUTOCOMPACT_BUFFER_TOKENS → autoCompact()          │   │
│  │    └─ If REACTIVE_COMPACT enabled → try reactiveCompact() │   │
│  │                                                               │   │
│  │ 2. buildQueryConfig()  // First iteration only                │   │
│  │                                                               │   │
│  │ 3. apiClient.createMessage({                                  │   │
│  │      model: config.model,                                     │   │
│  │      system: systemPrompt,                                    │   │
│  │      messages: normalizeMessagesForAPI(messages),             │   │
│  │      max_tokens: maxOutputTokensOverride,                     │   │
│  │      tools: buildToolDefinitions(config.tools),               │   │
│  │    })                                                         │   │
│  │                                                               │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              │                                      │
│                              ▼                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ for await (const event of streamResponse)                     │   │
│  │                                                               │   │
│  │  case 'text':                                                 │   │
│  │    yield { type: 'text', content: delta }                     │   │
│  │                                                               │   │
│  │  case 'tool_use':                                             │   │
│  │    ├─ StreamingToolExecutor.addTool(block, assistantMsg)  │   │
│  │    └─ yield { type: 'pending_tool_use' }  // Hover preview │   │
│  │                                                               │   │
│  │  case 'message_start':                                        │   │
│  │    ├─ yield { type: 'message_start', ttftMs }             │   │
│  │    └─ pushApiMetricsEntry(ttftMs)  // Track TTFT          │   │
│  │                                                               │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              │                                      │
│                              ▼                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
```

```
│  │ toolExecutor.getRemainingResults()                          │   │
│  │  ├─ processQueue()  // Scan executable tools                │   │
│  │  ├─ executeTool()   // Execute single tool                 │   │
│  │  ├─ Permission check → allow / deny / prompt                │   │
│  │  ├─ Concurrency safety → parallel / serial                 │   │
│  │  └─ yield { message: toolResult }  // Append to messages[]│   │
│  └──────────────────────────────────────────────────────────┘   │
│                              │                                    │
│                              ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ Decision point: Continue loop?                              │   │
│  │                                                             │   │
│  │ Condition 1: New tool calls? (StreamingToolExecutor.tools │   │
│  │   has new completions)                                      │   │
│  │   └─ Yes → Append tool_result to messages → Back to      │   │
│  │         apiClient.createMessage() [Continue loop]          │   │
│  │                                                             │   │
│  │ Condition 2: max_output_tokens error?                       │   │
│  │   └─ Yes → maxOutputTokensRecoveryCount++                  │   │
│  │         If < 3 → ESCALATED_MAX_TOKENS → Back to            │   │
│  │         apiClient                                           │   │
│  │         Otherwise → Return error_max_turns                 │   │
│  │                                                             │   │
│  │ Condition 3: tokenBudget exceeded?                          │   │
│  │   └─ Yes → budgetTracker.check() → continue / stop         │   │
│  │                                                             │   │
│  │ Condition 4: maxTurns reached?                              │   │
│  │   └─ Yes → Return error_max_turns                          │   │
│  │                                                             │   │
│  │ Condition 5: API error (retryable)?                         │   │
│  │   └─ Yes → yield SDKStatus('retrying') → retry             │   │
│  │                                                             │   │
│  │ Condition 6: No new tool calls (API returned final text)   │   │
│  │   └─ Yes → Append assistantMessage → Return                │   │
│  │         Terminal{done}                                      │   │
│  └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
                          Terminal { done }
                                   │
                                   ▼
                     notifyCommandLifecycle('completed')
                     return terminal
```

20.2 Tool Parallel/Serial Decision Flow

```
API returns N ToolCalls
        │
        ▼
┌──────────────────────────────────────┐
│ findToolByName(tools, block.name)  │
│  ├─ Tool exists → check inputSchema│
```

```
│  └─ Tool doesn't exist → return "No such tool" error │
└──────────────────────┬──────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────┐
│  isConcurrencySafe evaluation            │
│   ├─ parsed = tool.inputSchema.safeParse(input)│
│   ├─ If parse fails → isConcurrencySafe = false│
│   └─ If parse succeeds → tool.isConcurrencySafe(parsed.data)│
│       └─ Throws → isConcurrencySafe = false│
└──────────────────────┬──────────────────────────────┘
                       │
                       ▼
           ┌───────────┴──────────┐
           │                      │
           │  isConcurrencySafe   │  !isConcurrencySafe
           │       = true         │      = true
           │                      │
           ▼                      ▼
┌────────────────┐     ┌────────────────────┐
│ Enter Concurrent│    │ Enter Non-Concurrent│
│  execution group│    │   exclusive group  │
└────────┬───────┘     └──────────┬─────────┘
         │                        │
         ▼                        ▼
┌────────────────┐     ┌────────────────────┐
│ Check execution│     │  Wait for current  │
│ slot           │     │  execution group   │
│ Slot available?│     │  to complete       │
│  ├─ Yes →      │    │  → processQueue() │
│  │   Execute tool│   │    break scan loop│
│  └─ No →       │     └──────────┬─────────┘
│    Skip this   │                │
│  iteration     │                ▼
│  (retry later)│     ┌────────────────────┐
└────────┬───────┘     │ executeTool()      │
         │             │ (execute immediately)│
         │             └──────────┬─────────┘
         │                        │
         └────────────────────────┴──────────────┘
                                  │
                                  ▼
                     ┌────────────────────────┐
                     │ tool.status = 'completed' │
                     │ results → append to messages│
                     │ processQueue() continue scan│
                     └────────────────────────┘
```

20.3 Multi-Agent Coordination Flow

```
Coordinator receives task T
        │
        ▼
```

```
Task Decomposition (T → {S1, S2, S3, ..., Sn})
        │
        ▼
┌──────────────────────────────────────────┐
│ Dependency analysis → Build parallel groups│
│                                           │
│  Group 1: [S1, S2]  // No dependencies, parallel│
│  Group 2: [S3]      // Depends on Group 1   │
│  Group 3: [S4, S5]  // Depends on Group 2   │
└──────────────────────┬────────────────────┘
                       │
        ┌──────────────┴──────────────┐
        ▼                             ▼
  ┌──────────────┐            ┌──────────────┐
  │ spawn Agent()│            │ spawn Agent()│
  │ for S1       │            │ for S2       │
  │ subagent_type│            │ subagent_type│
  │ = "worker"   │            │ = "worker"   │
  └──────┬───────┘            └──────┬───────┘
         │                           │
         ▼                           ▼
  ┌──────────────────┐      ┌──────────────────┐
  │ Worker 1         │      │ Worker 2         │
  │ query() loop     │      │ query() loop     │
  │ tools = Worker   │      │ tools = Worker   │
  │   Allowed Tools  │      │   Allowed Tools  │
  └──────────┬───────┘      └──────────┬───────┘
             │                         │
             └─────────────────────────┘
                          │
                          ▼
             <task-notification> XML
                          │
                          ▼
             TaskOutput collection → aggregateResult()
                          │
                          ▼
             Coordinator analyzes results
                          │
           ┌──────────────┴──────────────┐
           │  Group 1 done?              │
           │  ├─ Yes → Launch Group 2│
           │  └─ No → Wait               │
           └─────────────────────────────┘
                          │
                          ▼
             Repeat until all groups done
                          │
                          ▼
             synthesizeResults(allResults)
                          │
                          ▼
             Return final report to user
```

21. Summary

21.1 Agent Algorithm Core Points

1. **AsyncGenerator Pattern**: The entire system's streaming processing is based on TypeScript Generator + `yield*` delegation. This allows `query()` to transparently stream forward API events while injecting intermediate logic when needed (e.g., permission checks, compaction).
2. **Two-Layer Tool Orchestration**:
   - **API layer** (API returns multiple ToolCalls at once): `StreamingToolExecutor` decides parallel/serial execution
   - **Execution layer** (tools may continue to trigger more ToolCalls): Forms a recursive loop until API no longer requests tools
3. **Token Budget as First-Class Citizen**: From `getAutoCompactThreshold` (precise threshold calculation to the single digit) to the circuit breaker design with `MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3`, the entire system operates around the token budget.
4. **Layered Agent Architecture**:
   - **Main Agent**: User's main conversation, has all tools
   - **LocalAgent**: Subprocess isolation, file system-level security
   - **RemoteAgent**: Crosses CCR remote boundary
   - **InProcessTeammate**: Zero-overhead memory-level agent
   - **Coordinator**: Meta-agent managing multiple workers
5. **Compaction as a Service**: Context compaction is not just memory optimization, but a precise token economics implementation — preserving the 5 most recent files (5000 tokens/file), 5 skills (5000 tokens/skill), with the rest of history replaced by summaries.
6. **Memory as Behavioral Guidance**: The memory system (user/feedback/project/reference four types) is not simple storage; it requires `**Why:**` and `**How to apply:**` fields, giving memories executable context.

21.2 Key File Index

File	Lines	Core Responsibility
`query.ts`	~800+	`query()` async generator + `queryLoop()` state machine
`QueryEngine.ts`	~1300+	`QueryEngine` class, API wrapper, conversation management
`tools/AgentTool/AgentTool.tsx`	~1400+	Sub-agent launch, validation, async registration
`tools/AgentTool/runAgent.ts`	~1000+	Local agent execution engine
`services/tools/StreamingToolExecutor.ts`	~500+	Tool parallel/serial orchestration engine
`coordinator/coordinatorMode.ts`	~400+	Coordinator system prompts
`services/compact/autoCompact.ts`	~350+	Auto Compact algorithm, precise thresholds
`services/compact/compact.ts`	~350+	Compaction execution core algorithm
`tasks/LocalAgentTask/LocalAgentTask.tsx`	~600+	Async agent lifecycle management
`state/AppStateStore.ts`	~500+	Global React state management
`memdir/memdir.ts`	~300+	Memory system (storage + format)
`tools/AgentTool/builtInAgents.ts`	~70	Built-in agent dynamic registration

File	Lines	Core Responsibility
`tools/AgentTool/built-in/generalPurposeAgent.ts`	~35	Default agent definition
`query/tokenBudget.ts`	~100+	Token budget tracking
`constants/prompts.ts`	~200+	System prompt constants
`tools/AgentTool/constants.ts`	~15	Agent constant definitions

21.3 Notable Engineering Designs

Design Pattern	Application Scenario	Implementation Location
Generator Async Flow	API streaming / tool result streaming	`query.ts`
Dependency Injection	Dependency injection (testability)	`query/deps.ts`
Feature Flag	A/B testing, gradual rollout	Statsig GrowthBook
Circuit Breaker	Compaction failure circuit breaker	`autoCompact.ts`
Child AbortController	Hierarchical cancellation (parent/child agents)	`runAgent.ts`